

TECHNICAL INTERVIEW REFERENCE

.NET Developer Interview Prep Guide

35 Essential Topics for 3-Year .NET Engineers

A consolidated, exam-style reference covering C#, the .NET runtime, ASP.NET Core, EF Core, async & multithreading, design patterns, SOLID, microservices, Docker, Kubernetes, Azure, and common interview questions — each with definitions, code examples, and trade-offs you can recite in 60 seconds.

C# & CLR

ASP.NET Core

EF Core

Async

LINQ

DI

Microservices

Docker

K8s

Azure

SOLID

Design Patterns

System Design

EXPERIENCE
LEVEL**3 Years**TOPICS
COVERED**35 Sections**

FORMAT

**Concept + Code +
Q&A**

Table of Contents

Foreword	8
1. Introduction to .NET	9
1.1 Key Components	9
1.2 .NET Framework vs .NET Core vs .NET 5+	9
1.3 Compile & Execution Pipeline	9
2. C# Fundamentals	10
2.1 Value Types vs Reference Types	10
2.2 Type Inference & var	10
2.3 Strings & StringBuilder	11
2.4 Nullable Reference Types (C# 8+)	11
2.5 Pattern Matching (modern C#)	11
2.6 Records (C# 9+)	11
3. Object-Oriented Programming (OOP)	12
3.1 The Four Pillars	12
3.2 Access Modifiers	12
3.3 Abstract Class vs Interface	12
3.4 Virtual / Override / new	13
3.5 Static, Sealed, Partial	13
4. Advanced C# Concepts	13
4.1 ref / out / in / ref readonly	13
4.2 Span and Memory	14
4.3 ref return and ref struct	14
4.4 Tuples and Deconstruction	14
4.5 Extension Methods	14
4.6 Pattern Matching (deep dive)	15
4.7 Source Generators (C# 9+)	15
5. Collections & Generics	15
5.1 Generic Collections Cheat Sheet	15
5.2 How List Grows	15
5.3 Dictionary Internals	16
5.4 IEnumerable vs IQueryable vs IList	16

5.5 Generics: Constraints	16
5.6 Covariance and Contravariance	16
6. Exception Handling	17
6.1 Exception Hierarchy	17
6.2 Try / Catch / Finally Semantics	17
6.3 Custom Exceptions	18
6.4 Exception-throwing Patterns	18
7. Delegates, Events & Lambda Expressions	18
7.1 Delegate Types	18
7.2 Delegate vs Event	19
7.3 Multicast Delegates	19
7.4 Lambda Expressions	19
7.5 Closures and Captured Variables	19
7.6 Delegate vs Expression Tree	20
8. LINQ (Language Integrated Query)	20
8.1 Syntax: Query vs Method	20
8.2 Most-Used Operators	20
8.3 Deferred vs Immediate Execution	21
8.4 LINQ to Objects vs LINQ to Entities	21
8.5 GroupBy Example	21
8.6 Performance Pitfalls	21
9. Asynchronous Programming	22
9.1 async / await Basics	22
9.2 How async/await Actually Works	22
9.3 Returning Task vs void	22
9.4 CancellationToken	23
9.5 Common Pitfalls	23
9.6 Task.Run and CPU-bound Work	23
9.7 WhenAll, WhenAny, WhenEach	23
10. Memory Management & Garbage Collection	24
10.1 GC Generations	24
10.2 GC Workflow	24
10.3 Dispose Pattern & IDisposable	24
10.4 Implementing IDisposable (with SafeHandle)	25

10.5 Finalizers	25
10.6 Memory Leaks in Managed Code	25
10.7 Tools	25
11. .NET Runtime & CLR	26
11.1 JIT Compilation	26
11.2 Assembly Loading	26
11.3 The Type System	26
11.4 Thread Pool	26
11.5 Server GC vs Workstation GC	27
12. ASP.NET Core	27
12.1 Program.cs (Minimal Hosting Model)	27
12.2 Request Pipeline & Middleware	27
12.3 Dependency Injection (Built-in)	28
12.4 Routing	28
12.5 Configuration & Options	29
12.6 Hosting Models	29
13. ASP.NET Core Web API	29
13.1 REST Principles	29
13.2 HTTP Verb Semantics	30
13.3 Controller Conventions	30
13.4 Status Codes You Should Know	31
13.5 Versioning	31
13.6 Filters	31
13.7 OpenAPI / Swagger	32
14. Entity Framework Core	32
14.1 DbContext Lifecycle	32
14.2 Code-First Workflow	32
14.3 Loading Related Data	33
14.4 Concurrency	33
14.5 Transactions	33
14.6 Performance Tips	34
15. Authentication & Authorization	34
15.1 JWT (JSON Web Token)	34
15.2 Token Generation	34

15.3 Refresh Tokens	35
15.4 Authorization Policies	35
15.5 Other Common Schemes	35
16. Dependency Injection	36
16.1 Constructor Injection (Preferred)	36
16.2 Registration Patterns	36
16.3 Keyed Services (.NET 8+)	36
16.4 Scrutor (Assembly Scanning)	37
16.5 Anti-Patterns	37
17. Middleware (Deep Dive)	37
17.1 Pipeline Order	37
17.2 Three Ways to Write Middleware	38
17.3 Branching the Pipeline	38
17.4 Common Custom Middleware Examples	38
17.5 Endpoint Routing vs Classic Routing	38
18. Logging & Monitoring	39
18.1 ILogger Basics	39
18.2 Structured Logging with Serilog	39
18.3 Log Scopes	39
18.4 OpenTelemetry (Modern Observability)	40
18.5 Health Checks	40
19. Microservices	40
19.1 Monolith vs Microservices	40
19.2 Communication Patterns	41
19.3 Service Discovery	41
19.4 Patterns for Reliability	41
19.5 Polly Example	42
19.6 Data Consistency	42
20. Design Patterns	42
20.1 Creational Patterns	42
20.2 Structural Patterns	43
20.3 Behavioural Patterns	43
20.4 Singleton (Thread-safe)	43
20.5 Strategy Pattern	43

20.6 Repository & Unit of Work	44
20.7 CQRS with MediatR	44
21. SOLID Principles	44
21.1 S — Single Responsibility Principle (SRP)	44
21.2 O — Open/Closed Principle (OCP)	45
21.3 L — Liskov Substitution Principle (LSP)	45
21.4 I — Interface Segregation Principle (ISP)	45
21.5 D — Dependency Inversion Principle (DIP)	45
21.6 Worked Example	45
22. Unit Testing	46
22.1 Frameworks	46
22.2 AAA Pattern	46
22.3 Test Doubles	46
22.4 Mocking with Moq	47
22.5 What to Test (and What Not To)	47
22.6 Integration Tests	47
23. Caching	48
23.1 Cache Layers	48
23.2 IMemoryCache	48
23.3 IDistributedCache (Redis)	48
23.4 Cache Patterns	49
23.5 Expiration Policies	49
23.6 Cache Stampede Protection	49
23.7 Output Caching & Response Caching	49
24. SignalR	50
24.1 Hub Basics	50
24.2 Client (JavaScript)	50
24.3 Targets for Sending	51
24.4 Groups & Users	51
24.5 Streaming	51
24.6 Scaling Out	51
24.7 Alternatives	52
25. Docker & Kubernetes Basics	52
25.1 Dockerfile for ASP.NET Core	52

25.2 Essential Docker Commands	52
25.3 Kubernetes Resources You Should Know	53
25.4 Sample Deployment + Service	53
26. Azure Basics	54
26.1 Compute Options for .NET	54
26.2 Storage Options	54
26.3 Application Insights / OpenTelemetry	55
26.4 Key Vault & Managed Identity	55
26.5 Common Patterns	55
27. Multithreading	56
27.1 Thread vs Task vs ThreadPool	56
27.2 Thread Safety Primitives	56
27.3 lock Statement	57
27.4 SemaphoreSlim (async-friendly)	57
27.5 Parallel / PLINQ	57
27.6 Channels	58
27.7 Deadlock Causes & Prevention	58
28. Performance Optimization	58
28.1 Measure First	58
28.2 BenchmarkDotNet Example	59
28.3 Common Optimisations	59
28.4 Allocation-Aware Code	59
28.5 Async Performance	60
29. Security Best Practices	60
29.1 OWASP Top 10 (2021)	60
29.2 SQL Injection — Always Parameterise	61
29.3 XSS — Encode Output	61
29.4 CSRF	61
29.5 Password Storage	61
29.6 Secrets Management	62
29.7 HTTPS & TLS	62
30. REST & HTTP Concepts	62
30.1 HTTP Request/Response Anatomy	62
30.2 Important Headers	63

30.3 Status Code Categories	63
30.4 Idempotency	63
30.5 REST Maturity (Richardson Model)	64
30.6 REST vs gRPC vs GraphQL	64
30.7 Pagination Patterns	64
31. System Design Basics	64
31.1 Six-Step Framework	64
31.2 Key Concepts	65
31.3 Worked Example: Design a URL Shortener	65
33. Coding Problems	66
33.1 Two Sum	66
33.2 Reverse a Linked List	66
33.3 Valid Parentheses	66
33.4 Async Producer-Consumer with Channels	67
33.5 Thread-Safe LRU Cache	67
34. HR & Scenario-Based Questions	68
34.1 STAR Framework	68
34.4 Questions to Ask the Interviewer	68
35. Quick Revision Notes	68
35.1 C# & Runtime	68
35.2 ASP.NET Core	69
35.3 EF Core	69
35.4 Concurrency & Threading	69
35.5 Web & API	70
35.6 Patterns & SOLID	70
35.7 DevOps & Cloud	70
35.8 Final Tips	70

Foreword

This guide consolidates 35 essential topics that a .NET developer with three years of experience is expected to know in technical interviews and day-to-day work. Each topic is structured as a concise definition, the core concepts you must internalise, a short code or scenario example, common pitfalls, and one or two interview-style questions with model answers. The aim is to give you a single revision document that you can read end-to-end in a few hours and revisit before any interview round.

The material reflects the realistic expectations of mid-level .NET roles: solid C# and CLR fundamentals, ASP.NET Core and Entity Framework Core for web workloads, async and multithreading for performance, design patterns and SOLID for maintainable code, and a working knowledge of Docker, Azure, and microservices for modern delivery. The closing sections focus on common interview questions, coding problems, and a quick-revision sheet you can scan minutes before the interview.

How to use this guide: Read each topic in order; do not skip the code examples even if the concept feels familiar. Most interview rejections happen because a candidate can define a concept but cannot write it down. After finishing a topic, attempt to recite the example from memory and compare it with the printed version.

1. Introduction to .NET

.NET is a free, cross-platform, open-source developer platform for building many types of applications. It is the unified successor of .NET Framework (Windows-only), .NET Core (cross-platform rewrite), and Xamarin/Mono (mobile). With .NET 5+ (currently .NET 8 LTS and .NET 9), Microsoft merged these stacks into a single runtime, Base Class Library (BCL), and SDK, so the same code runs on Windows, Linux, macOS, iOS, Android, and WASM.

1.1 Key Components

- **Runtime (CoreCLR / Mono / NativeAOT)** — JIT- or AOT-compiled execution engine with GC, type safety, and exception handling. CoreCLR is the server runtime; Mono is used for mobile and WASM; NativeAOT compiles ahead-of-time for startup-critical scenarios.
- **Base Class Library (BCL)** — A large set of reusable types: collections, IO, networking, JSON, HTTP, threading, reflection. Same library on every platform.
- **SDK & CLI** — dotnet CLI (`dotnet new`, `dotnet build`, `dotnet run`, `dotnet publish`) is the canonical build experience. MSBuild remains the underlying project model.
- **Workloads** — ASP.NET Core (web), EF Core (ORM), ML.NET (ML), WinForms/WPF/UWP/MAUI (desktop & mobile), SignalR (real-time), gRPC, etc.
- **NuGet** — The package manager. Packages are .nupkg files containing compiled assemblies and metadata. dotnet restore resolves the dependency graph.

1.2 .NET Framework vs .NET Core vs .NET 5+

Version	Platform	Open Source	Side-by-side	Status / Use
.NET Framework 4.8	Windows only	No	No	Legacy / maintenance
.NET Core 3.1 (EOL)	Cross-platform	Yes	Yes	Migrated to .NET 5+
.NET 8 (LTS)	Cross-platform	Yes	Yes	Recommended for production
.NET 9 (STS)	Cross-platform	Yes	Yes	Latest features, 18-month support
.NET 10 (LTS, 2025)	Cross-platform	Yes	Yes	Upcoming LTS

1.3 Compile & Execution Pipeline

Source code (.cs) goes through three stages before it runs:

- **C# Compiler (Roslyn)** — Translates .cs files into Intermediate Language (IL) assemblies (.dll / .exe). IL is CPU-independent bytecode.
- **JIT Compiler (RyuJIT)** — At runtime, the JIT translates IL methods into native machine code on first call. Compiled methods are cached for the process lifetime.

- **Native AOT (optional)** — At publish time, IL is compiled directly to native code, producing a self-contained binary with no JIT. Trade-off: smaller startup & footprint, but no runtime code generation (reflection limited).

```
# Create and run a console app
dotnet new console -n HelloApp
cd HelloApp
dotnet run # build + run
dotnet publish -c Release -r linux-x64 --self-contained # standalone binary
```

2. C# Fundamentals

C# is a strongly-typed, object-oriented, garbage-collected language that targets .NET. This section covers the language primitives, value vs reference types, type inference, and the syntax features you must know cold for any mid-level interview.

2.1 Value Types vs Reference Types

Concept	Description
Value types	Stored inline where the variable is declared. Assignments copy the value. Includes primitives (int, double, bool, char), structs (DateTime, Guid), enums, and nullable value types (int?).
Reference types	Stored on the heap; the variable holds a reference. Assignments copy the reference, not the data. Includes classes, arrays, delegates, interfaces, and strings (strings are immutable reference types).
Boxing	Converting a value type to object or an interface. Allocates a heap object and copies the value — expensive in hot paths. Avoid with generics.
Unboxing	Extracting the value type from a boxed object. Requires an explicit cast and a type check at runtime.

```
int a = 10; // value type, on stack
int b = a; // copy: b == 10
b = 20; // a still 10

var p1 = new Point { X = 1, Y = 2 }; // class - reference
var p2 = p1; // p2 and p1 share object
p2.X = 99; // p1.X is now 99 too
```

2.2 Type Inference & var

The **var** keyword asks the compiler to infer the type from the right-hand side. It is syntactic sugar — the variable is still strongly typed at compile time. Use **var** when the type is obvious (right-hand side is a constructor or literal) and explicit types when the type adds clarity (e.g., LINQ projections returning anonymous shapes).

2.3 Strings & StringBuilder

- **string (System.String)** — Immutable reference type. Every concatenation creates a new object. Interning pools identical literal strings.
- **StringBuilder** — Mutable character buffer. Use for loops that build strings (6+ concatenations). Pre-size the Capacity to avoid reallocations.
- **String comparison** — Use StringComparison.Ordinal / OrdinalIgnoreCase for security and default behaviour; use CurrentCulture for UI strings; never use == for culture-aware comparisons.

```
var sb = new StringBuilder(capacity: 256);
for (int i = 0; i < 1000; i++)
    sb.Append(i).Append(','); // no intermediate strings
string result = sb.ToString();

// String equality
string.Equals(a, b, StringComparison.OrdinalIgnoreCase); // ✓ correct
a.ToLower() == b.ToLower(); // ✗ allocates
```

2.4 Nullable Reference Types (C# 8+)

Since C# 8, reference types can be annotated with ? to indicate nullability intent. string is non-nullable; string? is nullable. The compiler emits warnings if you dereference a nullable without checking. This is a compile-time feature — at runtime a reference is still just a reference.

```
#nullable enable
public class Customer {
    public string Name { get; set; } // non-nullable, must be set
    public string? Email { get; set; } // may be null
}

void Send(Customer c) {
    Console.WriteLine(c.Name.Length); // OK
    Console.WriteLine(c.Email.Length); // ⚠ warning
    if (c.Email is not null)
        Console.WriteLine(c.Email.Length); // OK
}
```

2.5 Pattern Matching (modern C#)

- **Switch expressions** — `return shape switch { Circle c => Math.PI*c.R*c.R, Square s => s.S*s.S, _ => 0 };`
- **Property patterns** — `if (p is { X: > 0, Y: > 0 }) ...` — concise null+property checks.
- **Tuple patterns** — Switch on multiple values without nested if/else.
- **List patterns (C# 11)** — Match sequences: `[first, .., last]` decomposes arrays/lists.

2.6 Records (C# 9+)

A record is a reference type with value-based equality semantics. Two record instances with the same values are equal. Records support `with` expressions (non-destructive mutation), positional constructors, and `record struct` (C# 10). They are ideal for DTOs and immutable domain models.

```
public record Customer(Guid Id, string Name, string? Email);

var c1 = new Customer(Guid.NewGuid(), "Alice", null);
```

```
var c2 = c1 with { Email = "alice@x.com" }; // c1 unchanged
bool same = c1 == c2; // true (value equality)
```

3. Object-Oriented Programming (OOP)

C# is a multi-paradigm language with first-class OOP support. The four pillars — Encapsulation, Inheritance, Polymorphism, Abstraction — are universally expected in interviews. Beyond definitions, you should be able to translate each pillar into C# keywords and design trade-offs.

3.1 The Four Pillars

Pillar	Definition	C# Mechanism
Encapsulation	Hide internal state; expose only via methods/properties	private fields + public properties; access modifiers
Inheritance	Derive a class from a base to reuse and extend behaviour	class Dog : Animal; sealed to prevent; base to call parent
Polymorphism	Same call site, different runtime behaviour	virtual + override; abstract methods; interfaces
Abstraction	Define "what" without "how"; separate contract from implementation	abstract classes; interfaces

3.2 Access Modifiers

Modifier	Accessibility	Typical Use
public	No restriction	API surface
private	Same class only (default for members)	Internal state
protected	Same class + derived classes	Hooks for subclasses
internal	Same assembly	Assembly-level helpers
protected internal	Same assembly OR derived (anywhere)	Framework base classes
private protected	Same assembly AND derived	Internal extension points

3.3 Abstract Class vs Interface

Aspect	Detail
Inheritance	Single base class only; can implement multiple interfaces. Abstract class can hold state, constructors, and concrete methods.

Aspect	Detail
Interface (modern)	Since C# 8, interfaces can have default method implementations, static members, and even virtual members. Still no instance state.
When to choose	Use abstract class when subclasses share state or non-trivial common logic (Template Method pattern). Use interface when you only need a contract and want classes to be able to implement multiple contracts.
Sealed	A sealed class cannot be inherited. Mark classes sealed by default — inlining and devirtualisation are then possible, improving performance.

3.4 Virtual / Override / new

```

public class Animal {
    public virtual string Speak() => "...";
}
public class Dog : Animal {
    public override string Speak() => "Woof"; // polymorphic
}
public class Cat : Animal {
    public new string Speak() => "Meow"; // shadowing - NOT polymorphic
}

Animal a = new Dog();
a.Speak(); // "Woof" - virtual dispatch

Animal c = new Cat();
c.Speak(); // "..." - new does NOT override

```

3.5 Static, Sealed, Partial

- **static class** — Cannot be instantiated; all members must be static. Used for utility classes (Math, Convert). Since C# 11, static interface members exist.
- **sealed class** — Cannot be inherited. Default to sealed for performance: the JIT can devirtualise calls and inline aggressively.
- **partial class/method** — Allows a class to span multiple files (designer + user code). Partial methods are hooks that may or may not be implemented.

4. Advanced C# Concepts

Beyond the basics, modern C# (versions 8 through 12) introduced features that change how production code is written. Interviewers expect familiarity with these and the ability to explain the trade-offs.

4.1 ref / out / in / ref readonly

Modifier	Semantics	Typical Use
ref	By reference, must be assigned before call	Interoperate with native APIs; perf
out	By reference, must be assigned inside method	TryParse pattern, multiple returns

Modifier	Semantics	Typical Use
in	By reference, read-only	Large readonly structs (avoid copy)
ref readonly	Returns by reference, caller cannot modify	Returning large struct fields
ref struct	Stack-only — cannot be boxed or captured	Span, ref structs

4.2 Span and Memory

Span is a stack-only struct that represents a contiguous region of memory — arrays, sub-arrays, stackalloc, or unmanaged memory — without allocating. It enables zero-allocation parsing and string manipulation. Memory is its heap-safe counterpart that can be stored in fields and used across async boundaries.

```
string text = "Hello, World!";
ReadOnlySpan span = text.AsSpan();
ReadOnlySpan hello = span.Slice(0, 5); // no allocation
if (hello.SequenceEqual("Hello".AsSpan())) { /* ... */ }

// Stack-allocated buffer
Span buf = stackalloc int[64];
buf[0] = 42;
```

4.3 ref return and ref struct

Methods can return values by reference (`ref return`). Combined with ref structs this enables high-performance APIs (System.Text.Json, System.Memory) where escaping a managed reference would be unsafe. The compiler enforces span safety rules at compile time.

4.4 Tuples and Deconstruction

```
public (string Name, int Age) GetPerson() => ("Alice", 30);

var (name, age) = GetPerson(); // deconstruction
var pair = GetPerson();
Console.WriteLine(pair.Name); // named member access
```

4.5 Extension Methods

Extension methods let you "add" methods to existing types without modifying them. They are static methods in a static class, with the first parameter prefixed by `this`. LINQ is built entirely on extension methods. They do not actually modify the type — the compiler rewrites the call to a static method invocation.

```
public static class StringExt {
    public static bool IsBlank(this string? s) => string.IsNullOrEmpty(s);
}

string? s = Console.ReadLine();
if (s.IsBlank()) return;
```

4.6 Pattern Matching (deep dive)

```
decimal fee = customer switch {
    { Tier: "VIP", Age: > 65 } => 0m,
    { Tier: "Premium" } => 9.99m,
    { Tier: "Standard", Age: < 18 } => 4.99m,
    _ => 14.99m
};
```

4.7 Source Generators (C# 9+)

Source generators run at compile time and produce additional C# source that gets compiled into the same assembly. They enable zero-reflection serialisation (System.Text.Json pre-generates (de)serialisers), compile-time regex, and dependency injection registration without runtime reflection. Generators are read-only: they can analyse Roslyn syntax but cannot modify user code.

5. Collections & Generics

System.Collections.Generic is the workhorse namespace. Knowing which collection to choose for which workload is a frequent interview topic. Generics provide type safety and performance (no boxing) at compile time.

5.1 Generic Collections Cheat Sheet

Type	Backing	Complexity	Best For
List	Dynamic array	O(1) add (amortised), O(n) insert/remove middle	General-purpose ordered list
Dictionary	Hash table	O(1) get/set/contains	Lookups by key
HashSet	Hash table (values only)	O(1) add/remove/contains	Unique items, set ops
SortedDictionary	Red-black tree	O(log n) all ops	Sorted-by-key map
SortedSet	Red-black tree	O(log n) all ops	Unique sorted items
Queue	Ring buffer	O(1) enqueue/dequeue	FIFO processing
Stack	Array	O(1) push/pop	LIFO / DFS
LinkedList	Doubly linked list	O(1) insert/remove at node	Frequent mid-list edits
ConcurrentDictionary	Lock-stripped hash	O(1) scaled	Multi-threaded lookups

5.2 How List Grows

List uses an internal array. When it is full, a new array of double the size is allocated and elements are copied (Array.Copy). The default initial capacity is 4. Pre-sizing via `new List(capacity)` avoids costly reallocations — important for large lists.

5.3 Dictionary Internals

- **Buckets + entries arrays** — Hash code % bucket count picks a bucket; collisions are stored as a linked list within the entries array.
- **Hash code** — GetHashCode() must be fast, well-distributed, and deterministic for equal objects. Do NOT use mutable fields for hash code.
- **Equality** — Uses EqualityComparer.Default. For custom types, override Equals + GetHashCode or implement IEquatable.
- **Resize** — When load factor exceeds ~72%, the dictionary resizes to next prime and re-hashes everything.

5.4 IEnumerable vs IQueryable vs IList

Interface	Capability	LINQ Provider	When to Use
IEnumerable	Forward-only iteration	In-memory (LINQ-to-Objects)	Most general, lazy
IQueryable	Expression tree	Translated by provider (EF Core)	DB-side filtering
ICollection	Count, Add, Remove	In-memory	Need to mutate
IList	Indexed access	In-memory	Random access by index
IReadOnlyList	Indexed read-only	In-memory	Public API surface

Critical EF Core tip: Returning IEnumerable from a repository pulls the whole table into memory before filtering. Return IQueryable so that .Where(), .Select() compose into the SQL query.

5.5 Generics: Constraints

```
public class Repository where T : class, IEntity, new() {
    public T Create() => new T(); // new() constraint lets us instantiate
}

// Available constraints:
// where T : SomeClass (reference type or specific base)
// where T : struct (value type, non-nullable)
// where T : class (reference type)
// where T : notnull (non-nullable ref or value)
// where T : unmanaged (non-nullable, no ref fields)
// where T : new() (has parameterless ctor)
// where T : IComparable (implements interface)
```

5.6 Covariance and Contravariance

Variance lets you use a more-derived (covariance, out) or less-derived (contravariance, in) type than specified. IEnumerable is covariant: IEnumerable can be assigned to IEnumerable. Action is contravariant: Action can be assigned to Action. Variance only applies to reference types and only to interfaces/delegates declared with

in/out.

6. Exception Handling

Exceptions are for exceptional conditions, not normal control flow. Throwing an exception is ~1,000× slower than a return statement. The framework provides structured handling with try/catch/finally, exception filters, and patterns for predictable error reporting.

6.1 Exception Hierarchy

Type	Meaning	Typical Handling
Exception	Base class for all exceptions	Catch only as last resort
SystemException	Base for system-thrown exceptions	Do not throw directly
ApplicationException	Historical base for user exceptions	Deprecated — derive from Exception
InvalidOperationException	Object state invalid for call	API misuse
ArgumentException	Invalid argument passed	Include paramName
NullReferenceException	Dereferenced null	Avoid — fix with null checks
IOException	IO failure (disk, network)	Catch around IO calls
TaskCanceledException	Task cancelled via CancellationToken	Cooperative cancellation

6.2 Try / Catch / Finally Semantics

- **Order matters** — Catch from most specific to most general. A catch for Exception will swallow everything including OOM and StackOverflow — usually wrong.
- **finally always runs** — Even on return, goto, or exception. Use it for cleanup (dispose, close). finally does NOT run on StackOverflowException or OutOfMemoryException in some hosts.
- **Exception filters** — `catch (Exception e) when (e.HResult == 0x80004005)` — the when clause is evaluated before the stack unwinds, allowing conditional catches without rethrowing.
- **Rethrow** — Use `throw;` (preserves stack trace). Avoid `throw e;` which resets the stack and loses the original call site.

```
try {
    await repo.SaveAsync(entity);
}
catch (DbUpdateException ex) when (ex.InnerException is SqlException sql && sql.Number
== 2627) {
    logger.LogWarning(ex, "Duplicate key");
    return Conflict("Duplicate");
}
catch (Exception ex) {
    logger.LogError(ex, "Save failed");
    throw; // preserve stack trace
}
```

```
finally {
    scope?.Dispose();
}
```

6.3 Custom Exceptions

Derive from `Exception`, suffix the name with `Exception`, and implement the three canonical constructors (default, message, message+inner, serialization). Use custom exceptions only when callers will catch them by type — otherwise use the built-in types.

```
[Serializable]
public class OrderValidationException : Exception {
    public string OrderId { get; }
    public OrderValidationException(string orderId, string message)
        : base(message) => OrderId = orderId;
    public OrderValidationException(string orderId, string message, Exception inner)
        : base(message, inner) => OrderId = orderId;
    protected OrderValidationException(SerializationInfo info, StreamingContext ctx)
        : base(info, ctx) => OrderId = info.GetString(nameof(OrderId))!;
}
```

6.4 Exception-throwing Patterns

- **Fail fast** — Throw `ArgumentException` at the top of public methods for bad inputs. Do not half-process and then fail.
- **Do not swallow** — Empty catch blocks hide bugs. At minimum, log the exception.
- **One exception per error** — Do not throw generic `Exception` with a message string — callers cannot distinguish error types.
- **Prefer Result for expected failures** — Validation, not-found, conflict are expected outcomes; return a `Result` object instead of throwing.

7. Delegates, Events & Lambda Expressions

Delegates are type-safe function pointers. Events are delegates with restricted invocation. Lambdas are anonymous methods that the compiler converts into delegates or expression trees. Together they underpin LINQ, async, and the entire callback model of .NET.

7.1 Delegate Types

Family	Signature	Returns	Use Case
Action	Action	Returns void	Side-effect callbacks
Func	Func	Returns a value	Projections, predicates
Predicate	Predicate	Returns bool	Filtering
Custom delegate	delegate void Foo(int x);	Named, with optional out/ref	Public API contracts

7.2 Delegate vs Event

```
public class Publisher {
    // public delegate: anyone can invoke (bad for pub/sub)
    public Action? OnMessage;

    // event: only the class can invoke; outsiders can only += and -=
    public event EventHandler? MessageReceived;

    public void Run() {
        OnMessage?.Invoke("Hi"); // OK
        MessageReceived?.Invoke(this, "Hi"); // OK – only inside class
    }
}

var p = new Publisher();
p.OnMessage += msg => Console.WriteLine(msg); // OK
p.OnMessage("Hacked"); // ✗ also OK – bad
p.MessageReceived += (s, msg) => Console.WriteLine(msg); // OK
// p.MessageReceived(...); ✗ compile error – events are encapsulated
```

7.3 Multicast Delegates

All delegates are multicast — they hold an invocation list of zero or more methods. += adds, -= removes. Invoking a multicast delegate calls each method in order. Exceptions in any handler stop the chain — wrap with GetInvocationList() if you need to handle per-subscriber errors.

7.4 Lambda Expressions

```
Func square = x => x * x;
Func add = (a, b) => a + b;
Action log = msg => Console.WriteLine($"{DateTime.Now} {msg}");
Func isEven = n => n % 2 == 0;

// Statement lambda (multi-line)
Func factorial = n => {
    int r = 1;
    for (int i = 2; i <= n; i++) r *= i;
    return r;
};
```

7.5 Closures and Captured Variables

A lambda can reference variables from its enclosing scope. The compiler generates a hidden class to hold those variables (a closure). Beware the loop-variable trap: in C# 5+ foreach iteration variables are correctly captured per-iteration, but for-loops with int i capture the same variable — use a local copy inside the loop body.

```
// ✗ Common bug – all lambdas share the same i
var actions = new List();
for (int i = 0; i < 3; i++)
    actions.Add(() => Console.WriteLine(i));
foreach (var a in actions) a(); // prints 3, 3, 3

// ✓ Fix – local copy per iteration
for (int i = 0; i < 3; i++) {
    int local = i;
    actions.Add(() => Console.WriteLine(local));
}
```

```
}
```

7.6 Delegate vs Expression Tree

A lambda assigned to Func becomes a compiled delegate (executable code). A lambda assigned to Expression> becomes an expression tree — a data structure that LINQ providers (EF Core) translate into SQL or another query language. Expression trees cannot contain statement bodies or assignments.

8. LINQ (Language Integrated Query)

LINQ provides a uniform query syntax over objects (LINQ to Objects), XML, datasets, EF Core (LINQ to Entities), and others. It is one of the most-used features in modern C# and a perennial interview topic. Mastering deferred execution and provider translation is the difference between a junior and mid-level understanding.

8.1 Syntax: Query vs Method

```
var adults1 = from p in people
               where p.Age >= 18
               orderby p.Name
               select p.Name;

var adults2 = people
    .Where(p => p.Age >= 18)
    .OrderBy(p => p.Name)
    .Select(p => p.Name);

// Both compile to identical IL; method syntax is more common in modern code.
```

8.2 Most-Used Operators

Category	Operators	Purpose
Filtering	Where, OfType	Returns subset
Projection	Select, SelectMany	Transforms / flattens
Ordering	OrderBy, ThenBy, Reverse	Sorts
Grouping	GroupBy, ToLookup	Bucket by key
Aggregation	Count, Sum, Min, Max, Average, Aggregate	Reduces to scalar
Element	First, FirstOrDefault, Single, ElementAt	Returns one item
Set	Distinct, Union, Intersect, Except	Set operations
Join	Join, GroupJoin	Inner/left joins
Materialisation	ToList, ToArray, ToDictionary, ToHashSet	Forces execution

8.3 Deferred vs Immediate Execution

- **Deferred** — Where, Select, OrderBy, Skip, Take — return IEnumerable/IQueryable and do NOT execute until enumerated.
- **Immediate** — ToList, ToArray, Count, First, Any, Sum — force enumeration and execute the query right now.
- **Re-enumeration** — Each enumeration of a deferred query re-runs it. If the source changes between enumerations, results differ. Materialise with ToList to freeze.

```
var query = numbers.Where(n => n > 10); // deferred
// query has not run yet

var list = query.ToList(); // runs now
var count = query.Count(); // runs again – separate enumeration
```

8.4 LINQ to Objects vs LINQ to Entities

When the source is IQueryable (EF Core DbSet), LINQ operators build an expression tree that EF Core translates into SQL. Some operators have no SQL equivalent (e.g., custom C# methods) and force client-side evaluation, which can be catastrophic for performance. EF Core logs warnings for client-eval operators; in EF Core 3+ it throws by default.

```
// ✓ Translates to SQL WHERE
var active = db.Users.Where(u => u.IsActive).ToList();

// ✗ Force client evaluation (would throw in EF Core 3+)
var bad = db.Users.Where(u => MyHelper.IsValid(u.Email)).ToList();

// ✓ Compose as SQL
var good = db.Users
    .Where(u => u.LastLogin > DateTime.UtcNow.AddDays(-30))
    .OrderByDescending(u => u.LastLogin)
    .Take(50)
    .Select(u => new { u.Id, u.Email })
    .ToList();
```

8.5 GroupBy Example

```
var byDept = employees
    .GroupBy(e => e.Department)
    .Select(g => new {
        Department = g.Key,
        Count = g.Count(),
        AvgSalary = g.Average(e => e.Salary),
        Top = g.OrderByDescending(e => e.Salary).First()
    });
```

8.6 Performance Pitfalls

- **Multiple enumeration** — Enumerating an IEnumerable twice may re-run the underlying query. Materialise with ToList if you need to reuse.
- **Count() > 0 vs Any()** — Any() stops at the first match; Count() iterates all. Use Any() for existence checks.

- **Large projections** — Selecting entire entities when you only need two columns wastes memory and bandwidth. Project to a DTO.
- **N+1 queries** — Looping over a list and querying inside the loop is the classic EF Core anti-pattern. Use Include or a single join query.

9. Asynchronous Programming

Async/await is the canonical concurrency model in modern .NET. It enables non-blocking IO-bound operations without spawning threads, dramatically improving throughput for web servers and UIs. Misuse, however, causes deadlocks, memory leaks, and unpredictable behaviour.

9.1 async / await Basics

- **async modifier** — Marks a method as async; allows await inside. Does NOT run on a background thread — async is about non-blocking, not threading.
- **await operator** — Suspends the method until the awaited Task completes; the thread is returned to the pool. The continuation after await may run on a different thread.
- **Task** — Represents an ongoing operation that produces a result (Task) or no result (Task). The unit of async composition.
- **ValueTask** — Lightweight struct used when the result is often available synchronously. Reduces allocations but has stricter usage rules.

```
public async Task GetUserAsync(int id) {
    var json = await httpClient.GetStringAsync($"/users/{id}");
    return JsonSerializer.Deserialize(json)!;
}

await using var db = new AppDbContext();
var user = await db.Users.FindAsync(id); // non-blocking
Console.WriteLine(user.Name); // runs after await
```

9.2 How async/await Actually Works

The compiler generates a state machine. Each await becomes a state transition: register a continuation, return the method (releasing the thread), and resume from the next state when the awaited task completes. The SynchronizationContext (UI apps, legacy ASP.NET) controls where continuations run; in console apps and ASP.NET Core the context is null, so continuations run on thread-pool threads.

9.3 Returning Task vs void

Return Type	Verdict	Notes
Task / Task	✓ Recommended	Compose with await
ValueTask / ValueTask	✓ For hot, often-sync paths	Reduces allocations
async void	✗ Only for event handlers	Caller cannot await; exceptions crash the process

9.4 CancellationToken

Cancellation is cooperative: pass a CancellationTokens through your async APIs and check it (or call `ThrowIfCancellationRequested`) inside long-running work. The caller cancels via `CancellationTokenSource.Cancel()`. All BCL async methods accept a token.

```
public async Task DownloadAsync(string url, CancellationToken ct) {
    using var resp = await httpClient.GetAsync(url, ct);
    ct.ThrowIfCancellationRequested();
    return await resp.Content.ReadAsStringAsync(ct);
}

using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(10));
try {
    string body = await DownloadAsync(url, cts.Token);
}
catch (OperationCanceledException) {
    Console.WriteLine("Timed out");
}
```

9.5 Common Pitfalls

- **.Result / .Wait()** — Blocking on async code causes thread-pool starvation in ASP.NET Core and deadlocks in UI apps. Always await.
- **async void** — Cannot be awaited; exceptions propagate to the SynchronizationContext and crash the process. Use only for event handlers.
- **forget to await** — Forgetting to await a Task (fire-and-forget) swallows exceptions and lets work run after the request ends. Use `_ = DoAsync()` to mark intent explicitly.
- **.ConfigureAwait(false)** — In library code, call after every await to avoid capturing the SynchronizationContext. In ASP.NET Core and console apps it is a no-op but harmless.

9.6 Task.Run and CPU-bound Work

```
// IO-bound: just await – do NOT wrap in Task.Run
var data = await httpClient.GetByteArrayAsync(url);

// CPU-bound: offload to thread pool
var hash = await Task.Run(() => SHA256.HashData(bigBuffer));
```

9.7 WhenAll, WhenAny, WhenEach

- **Task.WhenAll** — Awaits all tasks; throws AggregateException if any fault (only first exception propagates — use `ExceptionDispatchInfo` to capture all).
- **Task.WhenAny** — Returns the first task to complete; useful for timeouts and races.
- **Task.WhenEach (NET 9)** — Streams tasks as they complete — cleaner than `WhenAny` loops.

```
var tasks = urls.Select(DownloadAsync).ToList();
try {
    var results = await Task.WhenAll(tasks); // parallel
}
catch (Exception ex) {
    // observe ALL exceptions to prevent UnobservedTaskException
    foreach (var t in tasks.Where(t => t.IsFaulted))
        logger.LogError(t.Exception, "Download failed");
}
```

10. Memory Management & Garbage Collection

.NET is garbage-collected: you allocate without freeing, and the GC reclaims memory it can prove is unreachable. Understanding the GC heap generations, finalisation, and the Dispose pattern is essential for writing efficient, leak-free code — especially in long-running services.

10.1 GC Generations

Generation	Contents	Collection Frequency	Typical Use
Gen 0	Short-lived, freshly allocated	Frequent, fast (sub-ms)	Locals, temp
Gen 1	Buffer between short and long-lived	Infrequent	Survivors of Gen 0
Gen 2	Long-lived objects	Rare, expensive (full GC)	Statics, singletons, caches
LOH (Large Object Heap)	Objects $\geq 85,000$ bytes	Compacted in .NET 5+	Big arrays, strings
POH (Pinned Object Heap)	Pinned objects (interop)	Never compacted	Pinned buffers

10.2 GC Workflow

- **Mark phase** — GC walks from roots (statics, locals, CPU registers) and marks every reachable object.
- **Plan phase** — GC decides whether to compact (move objects) or sweep (free in place).
- **Relocate / Sweep** — Live objects are moved (gen 0/1/2) or freed (LOH pre-.NET 5).
- **Promotion** — Survivors of a Gen 0 collection are promoted to Gen 1; Gen 1 survivors go to Gen 2.

10.3 Dispose Pattern & IDisposable

The GC reclaims memory but not unmanaged resources (file handles, DB connections, sockets). Types holding such resources implement IDisposable. The caller disposes them in a finally block — most easily via the using statement. In modern C#, `using` is a statement OR a declaration, and works with ref structs (e.g., `Span`).

```
// Modern using-declaration (no braces needed)
using var conn = new SqlConnection(cs);
await conn.OpenAsync();
// conn disposed at end of scope

// Classic using-statement (still valid)
using (var stream = File.OpenRead("data.bin")) {
    // use stream
} // stream disposed here
```

10.4 Implementing IDisposable (with SafeHandle)

```
public sealed class FileWriter : IDisposable {
    private readonly FileStream _stream;
    private bool _disposed;

    public FileWriter(string path) => _stream = File.OpenWrite(path);

    public void Write(byte[] data) {
        ObjectDisposedException.ThrowIf(_disposed, this);
        _stream.Write(data);
    }

    public void Dispose() {
        if (_disposed) return;
        _stream.Dispose();
        _disposed = true;
        GC.SuppressFinalize(this);
    }
}
```

10.5 Finalizers

A finalizer (~MyClass in C#) is called by the GC before reclaiming memory, giving the object a chance to release unmanaged resources. Finalizers are expensive: they push the object to the next GC generation and run on a dedicated thread. Modern best practice is to avoid finalizers and use SafeHandle (which wraps the unmanaged handle and finalises correctly).

10.6 Memory Leaks in Managed Code

- **Event handler subscriptions** — A long-lived publisher holding a reference to a short-lived subscriber via an event prevents GC. Unsubscribe in Dispose, or use weak event patterns.
- **Static caches** — Adding to a static dictionary and never removing keeps objects alive forever. Use MemoryCache with expiration or WeakReference.
- **Captured locals in lambdas** — A long-lived delegate capturing a closure keeps the closed-over variables alive.
- **Timer callbacks** — System.Timers.Timer holds a reference to its Elapsed handler; dispose the timer when done.
- **WPF / WinForms data binding** — Binding sources hold references to targets; clean up explicitly.

10.7 Tools

- **dotnet-counters** — Real-time GC stats: gen sizes, % time in GC, allocation rate.
- **dotnet-dump + SOS** — Heap dump analysis: !dumpheap, !gcroot to find leaks.
- **PerfView** — CPU + heap profiling, ETW events.
- **Visual Studio Memory Profiler / dotMemory** — Snapshot diff, retention paths.

11. .NET Runtime & CLR

The Common Language Runtime (CoreCLR in modern .NET) is the execution environment. It provides the type system, JIT compiler, garbage collector, thread pool, and security model. A mid-level developer should know the JIT, assembly loading, and AppDomain basics (or rather, the absence of AppDomains in modern .NET).

11.1 JIT Compilation

- **Tiered compilation (default since .NET 3)** — Methods are first JIT-compiled quickly with minimal optimisation (tier 0). When called frequently, they are recompiled with full optimisation (tier 1). Reduces startup while preserving steady-state throughput.
- **ReadyToRun (R2R)** — Pre-compiled IL to native code shipped in the assembly. Faster startup, larger binary, still falls back to JIT for un-precompiled methods.
- **NativeAOT** — Compile at publish time to a self-contained native binary. No JIT at runtime. Best for short-lived CLI tools and containers; limited reflection and dynamic loading.
- **Profile-Guided Optimisation (PGO)** — Dynamic PGO uses runtime profiling to specialise hot methods (e.g., inlining virtual calls when one implementation dominates). Enabled by default in .NET 6+.

11.2 Assembly Loading

Assemblies are loaded on demand by AssemblyLoadContext (ALC). Each ALC is a sandbox for assembly resolution; unloading an ALC releases all its assemblies. Modern .NET has no AppDomains — use AssemblyLoadContext for plugin scenarios. The default ALC shares the process-wide TPA (Trusted Platform Assemblies) list.

11.3 The Type System

Type Kind	Memory Model	Example
Reference type	Heap-allocated, GC-managed, can be null	class, array, string, delegate
Value type	Inline (stack for locals), copied on assignment	struct, enum, primitive
Boxed value	Heap-allocated wrapper around a value type	object o = 5;
Generic type	Each closed generic has its own MethodTable	List vs List
ref struct	Stack-only, cannot be boxed	Span, ReadOnlySpan

11.4 Thread Pool

The .NET thread pool (ThreadPool) provides worker threads for Task.Run and IO completion ports for async IO. It grows and shrinks based on demand; the Work Stealing algorithm balances work across worker threads. ThreadPool.SetMinThreads controls the floor — raising it can eliminate the one-thread-per-second ramp-up delay under burst load.

11.5 Server GC vs Workstation GC

Mode	Architecture	Use Case	Trade-off
Workstation GC	1 heap, 1 GC thread	Default for client apps	Lower memory
Server GC	1 heap per logical CPU, parallel GC threads	Default for ASP.NET Core	Higher throughput
Background GC	Gen 2 collections run concurrently with mutator	Default in both modes	Lowest pause

12. ASP.NET Core

ASP.NET Core is the cross-platform web framework that replaced ASP.NET (System.Web). It is modular (everything is a NuGet package), high-performance (Kestrel server, often faster than Node.js and Go frameworks), and unified (MVC, Web API, Razor Pages, SignalR, Blazor all share the same pipeline).

12.1 Program.cs (Minimal Hosting Model)

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddDbContext(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("Default")));
builder.Services.AddScoped();

var app = builder.Build();

if (app.Environment.IsDevelopment()) {
    app.UseDeveloperExceptionPage();
}
else {
    app.UseExceptionHandler("/error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

12.2 Request Pipeline & Middleware

Each HTTP request flows through a chain of middleware components, each of which can inspect, modify, or short-circuit the request. Order matters: exception handling first, then static files, then routing, then auth, then endpoints.

```
// Custom middleware via factory
app.Use(async (ctx, next) => {
    var sw = Stopwatch.StartNew();
    try {
```

```

        await next(); // call next middleware
    }
    finally {
        logger.LogInformation("{Path} took {Ms} ms",
            ctx.Request.Path, sw.ElapsedMilliseconds);
    }
});

// Class-based middleware (DI-friendly)
public class CorrelationMiddleware {
    private readonly RequestDelegate _next;
    public CorrelationMiddleware(RequestDelegate next) => _next = next;
    public async Task Invoke(HttpContext ctx) {
        ctx.Items["CorrelationId"] = Guid.NewGuid().ToString();
        await _next(ctx);
    }
}

```

12.3 Dependency Injection (Built-in)

Lifetime	Scope	Use For
Singleton	One instance for the app lifetime	Stateless services, caches, configs
Scoped	One instance per request (scope)	DbContext, per-request state
Transient	New instance every time	Lightweight stateless helpers

Captive dependency: Never inject a Scoped or Transient service into a Singleton — the singleton captures the shorter-lived instance, making it effectively singleton and causing stale DbContexts. Either make the dependency singleton too, or inject `IServiceScopeFactory` and create a scope inside the singleton.

12.4 Routing

```

// Attribute routing (preferred for APIs)
[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase {
    [HttpGet("{id:int}")]
    public async Task< Get(int id) {
        var order = await _svc.GetAsync(id);
        return order is null ? NotFound() : Ok(order);
    }

    [HttpPost]
    public async Task< Create([FromBody] CreateOrder cmd) {
        var order = await _svc.CreateAsync(cmd);
        return CreatedAtAction(nameof(Get), new { id = order.Id }, order);
    }
}

```

12.5 Configuration & Options

```
// appsettings.json
{
  "ConnectionStrings": { "Default": "Server=.;Database=App;Trusted_Connection=True;"
},
  "Jwt": { "Issuer": "myapp", "Audience": "myapp", "SigningKey": "..." }
}

// Strongly-typed options
public class JwtOptions { public string Issuer { get; set; } = "";
                        public string Audience { get; set; } = "";
                        public string SigningKey { get; set; } = ""; }

builder.Services.Configure(builder.Configuration.GetSection("Jwt"));

public class JwtService(IOptions opts) {
    private readonly JwtOptions _o = opts.Value;
    // ...
}
```

12.6 Hosting Models

- **Kestrel (default)** — Cross-platform web server. Used alone or behind IIS/Nginx as a reverse proxy. Async, high-throughput.
- **IIS (out-of-process)** — ASP.NET Core Module forwards requests to Kestrel. Use for Windows-hosted enterprise deployments.
- **HTTP.sys (Windows-only)** — Windows-native server. Supports Windows authentication and HTTP/2 server push.

13. ASP.NET Core Web API

Web API is the REST framework built on ASP.NET Core. Since .NET Core 3+ it shares the same controllers, routing, and filters as MVC. With [ApiController] attribute, model validation, problem details, and attribute routing are enabled by convention.

13.1 REST Principles

Principle	Meaning	Benefit
Stateless	Each request contains all info; no server-side session	Scales horizontally
Uniform interface	HTTP verbs (GET/POST/PUT/PATCH/DELETE) + resource URIs	Predictable
Resource-oriented	URIs name resources (/orders/42), not actions (/getOrder?id=42)	Cacheable
HATEOAS (optional)	Responses include links to related actions	Discoverability
Layered	Client cannot tell if it talks to the server or an intermediary	CDN / proxy

13.2 HTTP Verb Semantics

Verb	Properties	Use	Cacheable
GET	Safe + Idempotent	Fetch resource	Yes (Cache-Control)
POST	Neither	Create resource / trigger action	No (unless explicit)
PUT	Idempotent	Replace whole resource	No
PATCH	Neither (in theory)	Partial update	No
DELETE	Idempotent	Remove resource	No

13.3 Controller Conventions

```
[ApiController] // auto 400, [FromBody] inferred, attribute routing
[Route("api/v1/{controller}")]
public class ProductsController(IProductService svc) : ControllerBase {

    [HttpGet]
    public async Task<> List(
        [FromQuery] int page = 1, [FromQuery] int size = 20)
        => Ok(await svc.ListAsync(page, size));

    [HttpGet("{id:guid}", Name = "GetProduct")]
    [ProducesResponseType(200)]
    [ProducesResponseType(404)]
    public async Task< Get(Guid id) {
        var p = await svc.GetAsync(id);
        return p is null ? NotFound() : Ok(p);
    }

    [HttpPost]
    [ProducesResponseType(201)]
    [ProducesResponseType(400)]
    public async Task< Create([FromBody] CreateProduct cmd) {
        var p = await svc.CreateAsync(cmd);
        return CreatedAtRoute("GetProduct", new { id = p.Id }, p);
    }

    [HttpPut("{id:guid}")]
    [ProducesResponseType(204)]
    public async Task Update(Guid id, [FromBody] UpdateProduct cmd)
        => await svc.UpdateAsync(id, cmd) ? NoContent() : NotFound();

    [HttpDelete("{id:guid}")]
    [ProducesResponseType(204)]
    public async Task Delete(Guid id)
        => await svc.DeleteAsync(id) ? NoContent() : NotFound();
}
```

13.4 Status Codes You Should Know

Code	When to Return
200 OK	Successful GET / PUT / DELETE
201 Created	Successful POST — include Location header
204 No Content	Successful PUT / DELETE with no body
400 Bad Request	Malformed / validation failure
401 Unauthorized	No / invalid authentication
403 Forbidden	Authenticated but not allowed
404 Not Found	Resource does not exist
409 Conflict	Duplicate / version conflict
422 Unprocessable Entity	Semantic validation failure (some APIs)
500 Internal Server Error	Unhandled server error
503 Service Unavailable	Temporarily down (retry-after)

13.5 Versioning

- **URI versioning** — /api/v1/products vs /api/v2/products. Most explicit, easy to cache. Use Microsoft.AspNetCore.Mvc.Versioning or manual route attributes.
- **Header versioning** — X-API-Version: 2.0. Cleaner URLs but harder to test in a browser.
- **Query string** — /api/products?api-version=2.0. Easy to default; mixes metadata with data.
- **Media type** — Accept: application/vnd.myapp.v2+json. RESTful but obscure for clients.

13.6 Filters

Filter Type	Timing	Interface	Example
Authorization	Before everything else	IAuthorizationFilter	[Authorize]
Resource	Before and after model binding	IResourceFilter	Caching
Action	Before and after action method	IActionFilter	Logging, validation
Exception	On unhandled exception	IExceptionHandler	Global error handler
Result	Before and after action result	IResultFilter	Response post-processing

13.7 OpenAPI / Swagger

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(o => {
    o.SwaggerDoc("v1", new OpenApiInfo { Title = "Orders API", Version = "v1" });
    var jwt = new OpenApiSecurityScheme {
        Name = "Authorization", Type = SecuritySchemeType.Http,
        Scheme = "bearer", BearerFormat = "JWT", In = ParameterLocation.Header
    };
    o.AddSecurityDefinition("bearer", jwt);
    o.AddSecurityRequirement(new OpenApiSecurityRequirement {
        { jwt, new[] { "bearer" } }
    });
});

if (app.Environment.IsDevelopment()) {
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

14. Entity Framework Core

EF Core is the official ORM for .NET. It supports LINQ queries, change tracking, migrations, and conventions over configuration. EF Core 8 added primitive collections, JSON columns, and HierarchyId; EF Core 9 added more SQL translation improvements.

14.1 DbContext Lifecycle

- **Scoped in ASP.NET Core** — One DbContext per HTTP request. All repositories share tracking and the same transaction.
- **Short-lived otherwise** — In console / background services, wrap in a using block or inject IDbContextFactory and create per unit-of-work.
- **Not thread-safe** — Never share a DbContext across threads. Use a factory or scoped registration.
- **Tracking on by default** — Queries track entities for change detection. Use AsNoTracking() for read-only queries to halve allocations.

14.2 Code-First Workflow

```
public class AppDbContext : DbContext {
    public DbSet<Product> Products => Set();
    public DbSet<Order> Orders => Set();

    protected override void OnModelCreating(ModelBuilder b) {
        b.Entity(e => {
            e.HasKey(p => p.Id);
            e.Property(p => p.Name).IsRequired().HasMaxLength(200);
            e.HasIndex(p => p.SKU).IsUnique();
            e.HasOne(p => p.Category)
                .WithMany(c => c.Products)
                .HasForeignKey(p => p.CategoryId)
                .OnDelete(DeleteBehavior.Restrict);
        });
    }
}
```

```
// Migrations
// dotnet ef migrations add InitialCreate
// dotnet ef database update
```

14.3 Loading Related Data

Strategy	API	SQL	When
Eager	.Include(o => o.Customer)	Single query with JOIN	You know you need it
Explicit	.Entry(order).Reference(o => o.Customer).Load()	Separate query on demand	Conditional
Lazy	virtual nav properties + UseLazyLoadingProxies()	Query on first access	Avoid — N+1 risk
Select	.Select(o => new OrderDto { Cust = o.Customer.Name })	Projected query	Best for read DTOs

N+1 problem: Loading 100 orders and then accessing order.Customer.Name triggers 101 SQL queries. Fix: use .Include() for eager loading or project with .Select() so the database returns only what you need in a single query.

14.4 Concurrency

```
public class Product {
    public Guid Id { get; set; }
    public string Name { get; set; } = "";
    public int Stock { get; set; }
    [Timestamp] public byte[] RowVersion { get; set; } = Array.Empty();
}

// On update, EF adds RowVersion to WHERE clause; if it changed (another user
// updated),
// 0 rows affected → DbUpdateConcurrencyException. Handle by reload + merge or 409
// response.
```

14.5 Transactions

```
using var tx = await db.Database.BeginTransactionAsync();
try {
    await db.Orders.AddAsync(order);
    await db.SaveChangesAsync();
    await inventory.ReserveAsync(order.Items);
    await tx.CommitAsync();
}
catch {
    await tx.RollbackAsync();
    throw;
}

// Or use ExecuteInTransactionAsync for atomic bulk operations
```

14.6 Performance Tips

- **AsNoTracking()** — Read-only queries skip change tracking — ~2× faster, half the memory.
- **AsSplitQuery()** — When Include-ing multiple collections, split into multiple SQL queries instead of one giant cartesian product.
- **Compile queries** — `CompileQuery.Compile()` pre-compiles a LINQ query for repeated use; useful when cold-start matters.
- **Bulk operations** — EF Core 7+ `ExecuteUpdate/ExecuteDelete` translate to `UPDATE/DELETE` without loading entities.
- **Avoid .ToList() too early** — Compose queries with `IQueryable`, materialise once at the end.

15. Authentication & Authorization

Authentication ("authn") verifies who the user is. Authorisation ("authz") verifies what they are allowed to do. ASP.NET Core ships with a flexible auth middleware that supports JWT, cookies, OAuth/OIDC, and custom schemes, plus a policy-based authorisation engine.

15.1 JWT (JSON Web Token)

A JWT is a self-contained token with three Base64-URL-encoded parts: header, payload, signature. The server signs it with a secret (HMAC) or private key (RSA/ECDSA). The client sends it in the `Authorization: Bearer` header. The server validates the signature and expiry; no database lookup needed for authn.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opts => {
        opts.TokenValidationParameters = new TokenValidationParameters {
            ValidIssuer = jwtOpts.Issuer,
            ValidAudience = jwtOpts.Audience,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwtOpts.SigningKey)),
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ClockSkew = TimeSpan.FromSeconds(30)
        };
    });

app.UseAuthentication(); // ⚠ before UseAuthorization
app.UseAuthorization();
```

15.2 Token Generation

```
public string IssueToken(User user) {
    var claims = new[] {
        new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
        new Claim(JwtRegisteredClaimNames.Email, user.Email),
        new Claim(ClaimTypes.Role, user.Role),
        new Claim("tenant", user.TenantId)
    };
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_opts.SigningKey));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
    var token = new JwtSecurityToken(
```

```

        issuer: _opts.Issuer, audience: _opts.Audience,
        claims: claims, expires: DateTime.UtcNow.AddMinutes(15),
        signingCredentials: creds);
    return new JwtSecurityTokenHandler().WriteToken(token);
}

```

15.3 Refresh Tokens

Access tokens are short-lived (5–15 minutes) to limit damage if stolen. Refresh tokens are long-lived (days/weeks), stored server-side (DB or Redis), and exchanged for new access tokens. On logout or compromise, revoke the refresh token. Rotation (issue a new refresh token on every use) defends against replay attacks.

15.4 Authorization Policies

```

builder.Services.AddAuthorization(opts => {
    opts.AddPolicy("AdminOnly", p => p.RequireRole("Admin"));
    opts.AddPolicy("Over18", p => p.Requirements.Add(new MinimumAgeRequirement(18)));
    opts.AddPolicy("TenantMember", p => p.Requirements.Add(new
TenantMemberRequirement()));
});

[Authorize(Policy = "Over18")]
[HttpGet("restricted")]
public IActionResult Restricted() => Ok();

// Custom requirement handler (DI-resolved)
public class MinimumAgeHandler : AuthorizationHandler {
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext ctx, MinimumAgeRequirement req) {
        var dob = ctx.User.FindFirstValue(ClaimTypes.DateOfBirth);
        if (dob is not null && DateTime.TryParse(dob, out var d)
            && DateTime.Today.Year - d.Year >= req.Age)
            ctx.Succeed(req);
        return Task.CompletedTask;
    }
}

```

15.5 Other Common Schemes

- **Cookie auth** — Browser session for server-rendered apps. Sliding expiration; CSRF protection via anti-forgery tokens.
- **OIDC (OpenID Connect)** — Identity layer on top of OAuth2. Use IdentityServer, Azure AD, Auth0, or Keycloak for centralised identity.
- **Windows / Negotiate** — Kerberos/NTLM for intranet apps. HTTP.sys only on Windows.
- **API Key** — Simple shared secret in header. Use API keys for service-to-service, never for end-user auth.

16. Dependency Injection

Dependency Injection is the realisation of the Dependency Inversion Principle: classes receive their dependencies through constructors (or properties/methods) rather than creating them. ASP.NET Core ships a built-in DI container; third-party containers (Autofac, Lamar, Scrutor) add advanced features like assembly scanning and interceptors.

16.1 Constructor Injection (Preferred)

```
public class OrderService {
    private readonly IOrderRepository _repo;
    private readonly IEmailSender _email;
    private readonly ILogger _logger;

    public OrderService(IOrderRepository repo, IEmailSender email,
        ILogger logger) {
        _repo = repo;
        _email = email;
        _logger = logger;
    }
}

// Modern primary constructor (C# 12)
public class OrderService(
    IOrderRepository repo, IEmailSender email, ILogger logger) { }
```

16.2 Registration Patterns

- **AddScoped()** — One instance per DI scope (per request in ASP.NET Core). Default for DbContext and per-request state.
- **AddTransient()** — New instance every time. Use for cheap, stateless helpers.
- **AddSingleton()** — Single instance app-wide. Use for caches, config, stateless services. Be careful: not thread-safe by default.
- **AddSingleton(instance)** — Register a pre-built instance.
- **AddKeyedScoped/Transient/Singleton** — .NET 8+ keyed services — register multiple implementations and select by key.
- **Factory registration** — AddScoped(sp => new T(...)) — full control over construction.

16.3 Keyed Services (.NET 8+)

```
builder.Services.AddKeyedTransient("email");
builder.Services.AddKeyedTransient("sms");

public class NotificationService(
    [FromKeyedServices("email")] INotifier email,
    [FromKeyedServices("sms")] INotifier sms) {
    // ...
}
```

16.4 Scrutor (Assembly Scanning)

```
// Register all implementations of IRepository in the assembly
builder.Services.Scan(scan => scan
    .FromAssemblyOf()
    .AddClasses(c => c.AssignableTo(typeof(IRepository<>))
        .Where(t => !t.IsAbstract))
    .AsMatchingInterface() // OrderRepository -> IOrderRepository
    .WithScopedLifetime());
```

16.5 Anti-Patterns

- **Service Locator** — Resolving services from `IServiceProvider` inside business logic. Hides dependencies; makes testing hard. Inject what you need.
- **Captive Dependency** — A singleton capturing a scoped/transient dependency. The captured instance lives forever, breaking its intended lifetime.
- **Mixed lifetimes** — A scoped service depending on a transient is fine; a transient depending on a singleton is fine. The reverse direction (singleton → scoped) is the captive bug.
- **Container as service** — Injecting `IServiceProvider` into controllers/services — usually indicates the design needs rework.

17. Middleware (Deep Dive)

Middleware is the request pipeline in ASP.NET Core. Each component receives the `HttpContext`, can act on it, and either calls the next middleware or short-circuits. The pipeline is built in `Program.cs` via `app.Use*`, `app.Map*`, and `app.Run`.

17.1 Pipeline Order

```
var app = builder.Build();

app.UseExceptionHandler("/error"); // 1. Catch-all errors
app.UseHsts(); // 2. HSTS header (production)
app.UseHttpsRedirection(); // 3. HTTP -> HTTPS
app.UseStaticFiles(); // 4. Serve wwwroot/
app.UseRouting(); // 5. Match routes
app.UseCors(); // 6. CORS preflight
app.UseAuthentication(); // 7. Identify user
app.UseAuthorization(); // 8. Enforce policies
app.UseSession(); // 9. (optional) session
app.MapControllers(); // 10. Endpoints
```

Order pitfall: `UseRouting` MUST come before `UseAuthentication/UseAuthorization`. `UseCors` MUST come before `UseAuthentication` (CORS preflight should not require auth). Wrong order = silent 401s or 404s.

17.2 Three Ways to Write Middleware

```
// 1. Inline delegate (quick, no DI)
app.Use(async (ctx, next) => {
    logger.LogInformation("→ {Path}", ctx.Request.Path);
    await next();
    logger.LogInformation("← {Status}", ctx.Response.StatusCode);
});

// 2. Factory-based (DI-friendly, no ctor needed)
app.UseMiddleware();

// 3. Convention-based (Use[ClassName] extension method)
public static class CorrelationExt {
    public static IApplicationBuilder UseCorrelation(this IApplicationBuilder app)
        => app.UseMiddleware();
}
```

17.3 Branching the Pipeline

```
// Map: branch on path prefix
app.Map("/health", health => {
    health.UseHealthChecks();
});

// MapWhen: branch on predicate
app.MapWhen(ctx => ctx.Request.Headers.ContainsKey("X-Mobile"),
    branch => branch.UseMobileResponseCompression());

// UseWhen: continue main pipeline after branch (no short-circuit)
app.UseWhen(ctx => ctx.Request.Path.StartsWithSegments("/api"),
    branch => branch.UseApiAuditLog());
```

17.4 Common Custom Middleware Examples

- **Correlation ID** — Add or read X-Correlation-Id; propagate to logging scope.
- **Request logging** — Method, path, status, duration; integrate with Serilog.
- **Rate limiting** — Built-in middleware in .NET 7+: `app.UseRateLimiter()` with policies.
- **Compression** — `app.UseResponseCompression()` — gzip/brotli.
- **IP safelist** — Reject requests from non-whitelisted IPs.
- **Tenant resolution** — Extract tenant from subdomain/header; set context item.

17.5 Endpoint Routing vs Classic Routing

Modern ASP.NET Core uses endpoint routing: routing decisions are made early in the pipeline (`UseRouting`), and endpoint metadata (auth policies, CORS, rate-limit policies) is available to middleware between `UseRouting` and the endpoint. This decouples routing from dispatch and enables metadata-driven cross-cutting concerns.

18. Logging & Monitoring

Logging is the primary observability signal in any production service. ASP.NET Core ships with a built-in ILogger abstraction; serious production deployments add structured logging (Serilog), metrics (OpenTelemetry / EventCounters), and distributed tracing (OpenTelemetry, Application Insights, Datadog).

18.1 ILogger Basics

```
public class OrderService(ILogger logger) {
    public async Task CreateAsync(CreateOrder cmd) {
        logger.LogInformation("Creating order for {UserId} with {ItemCount} items",
            cmd.UserId, cmd.Items.Count);
        // ...
        logger.LogInformation("Order {OrderId} created", order.Id);
        return order;
    }
}

// Log levels (in order):
// Trace = 0 - very detailed
// Debug = 1 - useful for developers
// Information = 2 - normal flow
// Warning = 3 - abnormal but handled
// Error = 4 - errors, exceptions
// Critical = 5 - unrecoverable
```

18.2 Structured Logging with Serilog

```
builder.Host.UseSerilog((ctx, lc) => lc
    .ReadFrom.Configuration(ctx.Configuration)
    .Enrich.FromLogContext()
    .Enrich.WithMachineName()
    .Enrich.WithEnvironmentName()
    .WriteTo.Console(new RenderedCompactJsonFormatter())
    .WriteTo.Seq("http://seq:5341")
    .WriteTo.Elasticsearch(new ElasticsearchSinkOptions(new Uri("http://es:9200")) {
        AutoRegisterTemplate = true, IndexFormat = "app-logs-{0:yyyy.MM.dd}"
    }));

// Use named placeholders (NOT string interpolation):
logger.LogInformation("User {UserId} ordered {Count} items", id, count); // ✓
structured
logger.LogInformation($"User {id} ordered {count} items"); // ✗ plain text
```

18.3 Log Scopes

```
// Scope adds contextual properties to all logs within the block
using (logger.BeginScope(new Dictionary {
    ["CorrelationId"] = correlationId,
    ["UserId"] = userId
})) {
    logger.LogInformation("Processing order");
    await ProcessOrderAsync();
    logger.LogInformation("Order done");
}
```

18.4 OpenTelemetry (Modern Observability)

- **Traces** — Distributed tracing across services. Each request gets a `TraceId`; spans capture work in each service.
- **Metrics** — Numeric measurements (counters, histograms, gauges) — request rate, duration, error rate.
- **Logs** — Structured log events correlated with traces via `TraceId`/`SpanId`.
- **Exporters** — Send to OTLP-compatible backends: Jaeger, Tempo, Prometheus, Application Insights, Datadog.

```
builder.Services.AddOpenTelemetry()
    .WithTracing(tp => tp
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddEntityFrameworkCoreInstrumentation()
        .AddOtlpExporter(o => o.Endpoint = new Uri("http://otel-collector:4317")))
    .WithMetrics(mp => mp
        .AddAspNetCoreInstrumentation()
        .AddRuntimeInstrumentation()
        .AddPrometheusExporter());

app.MapPrometheusScrapingEndpoint(); // /metrics for Prometheus
```

18.5 Health Checks

```
builder.Services.AddHealthChecks()
    .AddSqlServer(connectionString, name: "sql")
    .AddRedis(redisConnectionString, name: "redis")
    .AddUrlGroup(new Uri("https://api.upstream/health"), name: "upstream");

app.MapHealthChecks("/health", new HealthCheckOptions {
    ResponseWriter = UIResponseWriter.WriteHealthCheckUIResponse
});

// For load balancer: /health/live (process up) vs /health/ready (deps ready)
```

19. Microservices

A microservice architecture decomposes an application into small, independently deployable services that communicate over the network. It enables independent scaling, technology diversity, and team autonomy at the cost of operational complexity, network failures, and eventual consistency.

19.1 Monolith vs Microservices

Aspect	Monolith	Microservices
Deployment	Single unit	Independent per service
Scaling	Whole app	Per service
Tech stack	Uniform	Polyglot allowed
Data	Shared DB	DB per service

Aspect	Monolith	Microservices
Failure	Total outage	Partial (if isolated)
Communication	In-process method call	Network (HTTP/gRPC/message)
Operational complexity	Low	High
Team autonomy	Low	High

19.2 Communication Patterns

- **HTTP/REST** — Simple, ubiquitous, human-readable. Use for external-facing APIs and low-throughput internal calls.
- **gRPC** — Binary protocol over HTTP/2. Strongly-typed contracts via Protocol Buffers. Best for high-throughput internal service-to-service.
- **Message broker (RabbitMQ, Kafka, Azure Service Bus)** — Asynchronous, decoupled, durable. Use for events, work queues, fan-out.
- **Service mesh (Istio, Linkerd, Dapr)** — Sidecar proxy handles mTLS, retries, circuit breaking, observability — application code stays simple.

19.3 Service Discovery

- **DNS / load balancer** — Service name → single VIP. Simplest; works in Kubernetes with the kube-dns service.
- **Kubernetes Services** — ClusterIP / Headless Service with selector. Built-in load balancing and DNS.
- **Consul / Eureka** — External service registry. Services register on startup; clients query the registry. Common in non-Kubernetes setups.
- **Dapr** — Sidecar-based building block APIs (state, pub/sub, service invocation). Cloud-agnostic.

19.4 Patterns for Reliability

Pattern	Purpose	Implementation
Circuit Breaker	Trip on N failures; reject calls for a cooldown period	Polly
Retry with backoff	Retry transient failures; exponential backoff + jitter	Polly
Bulkhead	Limit concurrent calls to a downstream; isolate failures	Polly
Timeout	Always set; never wait indefinitely	HttpClient.Timeout, Polly
Idempotency keys	Safe retry of POST/PUT; deduplicate by key	Custom
Saga	Distributed transaction as a sequence of local txns + compensations	MassTransit, NServiceBus

19.5 Polly Example

```
var retry = Policy
    .Handle()
    .OrResult(r => r.StatusCode >= System.Net.HttpStatusCode.InternalServerError)
    .WaitAndRetryAsync(3, attempt => TimeSpan.FromSeconds(Math.Pow(2, attempt)));

var breaker = Policy
    .Handle()
    .CircuitBreakerAsync(5, TimeSpan.FromSeconds(30));

var timeout = Policy.TimeoutAsync(TimeSpan.FromSeconds(10));

var resilient = Policy.WrapAsync(retry, breaker, timeout);

var response = await resilient.ExecuteAsync(() =>
    httpClient.GetAsync("https://api.upstream/orders"));
```

19.6 Data Consistency

- **Database per service** — Each service owns its data; no shared schemas. Cross-service queries need composition at the API layer or a CQRS read model.
- **Saga pattern** — Long-running distributed transaction = sequence of local transactions. Each step publishes an event that triggers the next. On failure, run compensating actions.
- **Event sourcing** — Store every state change as an event; current state is the projection. Enables time travel and audit but adds complexity.
- **Outbox pattern** — Atomic DB write + event publish in the same transaction; a background publisher reads the outbox table and publishes to the broker. Ensures no lost events.

20. Design Patterns

Design patterns are reusable solutions to recurring design problems. The GoF (Gang of Four) catalogue 23 patterns; modern C# has idiomatic shortcuts for many. Interviewers expect you to recognise, name, and implement the most common ones.

20.1 Creational Patterns

Pattern	Intent	Example in .NET
Singleton	One instance app-wide	ILoggerFactory, IConfiguration
Factory Method	Subclass decides which class to instantiate	ILoggerProvider.CreateLogger
Abstract Factory	Family of related objects	DbProviderFactory
Builder	Step-by-step construction of complex objects	StringBuilder, IConfigurationBuilder
Prototype	Clone an existing object	ICloneable, MemberwiseClone

20.2 Structural Patterns

Pattern	Intent	Example
Adapter	Bridge incompatible interfaces	Stream → StreamReader
Bridge	Decouple abstraction from implementation	Abstraction + Implementor hierarchies
Composite	Tree of objects treated uniformly	FileSystem (file + folder)
Decorator	Add behaviour without subclassing	Stream subclasses (GZipStream)
Facade	Simplified API over complex subsystem	WebApplication.CreateBuilder
Proxy	Placeholder controlling access	Lazy, EF Core proxies

20.3 Behavioural Patterns

Pattern	Intent	Example
Strategy	Interchangeable algorithms behind one interface	IComparer, payment methods
Observer	Notify subscribers of state changes	Events, IObservable
Command	Encapsulate a request as an object	ICommand, Undo stacks
Template Method	Algorithm skeleton in base class, steps overridden	Stream.Read/Write
Iterator	Sequential access to a collection	IEnumerable
Chain of Responsibility	Pass request along a chain until handled	ASP.NET Core middleware
State	Behaviour changes with internal state	Workflow / state machines
Mediator	Central object coordinates colleagues	MediatR (CQRS)

20.4 Singleton (Thread-safe)

```
// Lazy – simplest thread-safe singleton
public sealed class Config {
    private static readonly Lazy<Config> _instance = new(() => new Config());
    public static Config Instance => _instance.Value;
    private Config() { /* load */ }
}

// DI Singleton (preferred in ASP.NET Core)
builder.Services.AddSingleton();
```

20.5 Strategy Pattern

```
public interface IPricingStrategy {
    decimal Calculate(Order order);
}
```

```
public class RegularPricing : IPricingStrategy {
    public decimal Calculate(Order o) => o.Subtotal;
}

public class DiscountPricing(decimal rate) : IPricingStrategy {
    public decimal Calculate(Order o) => o.Subtotal * (1 - rate);
}

public class CheckoutService(IPricingStrategy pricing) {
    public decimal Total(Order o) => pricing.Calculate(o);
}
```

20.6 Repository & Unit of Work

The Repository pattern abstracts data access behind a collection-like interface. The Unit of Work pattern tracks changes and commits them as one transaction. With EF Core, DbContext is already a Unit of Work and DbSet is a Repository — wrapping them in your own interfaces is mostly useful for testability or to hide EF-specific quirks. Modern practice often skips the wrapper and uses DbContext directly in tests with the SQLite in-memory provider.

20.7 CQRS with MediatR

```
// Commands and queries are separate objects
public record CreateOrderCommand(string Sku, int Qty) : IRequest;

public class CreateOrderHandler(AppDbContext db) : IRequestHandler {
    public async Task Handle(CreateOrderCommand cmd, CancellationToken ct) {
        var order = new Order(cmd.Sku, cmd.Qty);
        db.Orders.Add(order);
        await db.SaveChangesAsync(ct);
        return order.Id;
    }
}

// Controller is thin
[HttpPost]
public async Task Create([FromBody] CreateOrderCommand cmd)
    => await _mediator.Send(cmd);
```

21. SOLID Principles

SOLID is five principles of object-oriented design promoted by Robert C. Martin. Following them yields code that is easier to maintain, extend, and test. They are recited in nearly every mid/senior interview.

21.1 S — Single Responsibility Principle (SRP)

A class should have one, and only one, reason to change. Each class should solve one problem. A "CustomerService" that validates, persists, emails, and logs is doing too much. Split into CustomerValidator, CustomerRepository, EmailSender, AuditLogger.

21.2 O — Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification. Add new behaviour by adding new classes (Strategy, Decorator) rather than editing existing ones. Switch statements on type are a smell — replace with polymorphism or a strategy registry.

21.3 L — Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base types without altering correctness. A Square that inherits from Rectangle breaks LSP because setting Width on a Square changes Height too. The classic Microsoft violation: ReadOnlyCollection throws on Add — callers that depend on ICollection.Add cannot substitute it for a List.

21.4 I — Interface Segregation Principle (ISP)

Clients should not be forced to depend on methods they do not use. Prefer many small, focused interfaces over one fat interface. Instead of a single IFatService with 20 methods, split into IReadService, IWriteService, IAdminService. Implementations pick what they need.

21.5 D — Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions. In C# this means: depend on interfaces, not concrete classes; inject dependencies through constructors. The DI container wires up the implementation at composition root (Program.cs).

21.6 Worked Example

```
// BEFORE: violates SRP, OCP, DIP
public class OrderProcessor {
    public void Process(Order o) {
        if (o.Total > 1000) o.ApplyDiscount(0.1m); // rule hardcoded
        var db = new SqlDatabase(); // direct dependency
        db.Save(o);
        new SmtpClient().Send("orders@x.com", "New"); // SRP violation
    }
}

// AFTER: SRP + OCP + DIP
public class OrderProcessor(
    IOrderRepository repo, // DIP
    INotifier notifier, // DIP
    IEnumerable rules) { // OCP
    public async Task ProcessAsync(Order o) {
        foreach (var r in rules) r.Apply(o); // OCP: add rules without editing
        await repo.SaveAsync(o); // SRP: repo handles persistence
        await notifier.NotifyAsync(o); // SRP: notifier handles comms
    }
}
```

22. Unit Testing

Unit tests verify individual units (classes/methods) in isolation. They are the foundation of CI/CD pipelines, refactoring safety nets, and behaviour documentation. xUnit is the most popular framework in modern .NET; NUnit and MSTest remain common.

22.1 Frameworks

Framework	Characteristics	When to Use
xUnit	Modern, parallel by default, [Fact]/[Theory]	Recommended for new projects
NUnit	Feature-rich, [Test]/[TestCase]	Older codebases
MSTest	Microsoft-native, [TestMethod]	Enterprise legacy
TUnit (NET 9+)	Source-generator-based, fast	Emerging

22.2 AAA Pattern

```
public class OrderServiceTests {
    [Fact]
    public async Task CreateAsync_valid_command_returns_order_with_id() {
        // Arrange
        var repo = new Mock();
        repo.Setup(r => r.SaveAsync(It.IsAny()))
            .Callback(o => o.Id = Guid.NewGuid())
            .Returns(Task.CompletedTask);
        var svc = new OrderService(repo.Object);

        // Act
        var order = await svc.CreateAsync(new CreateOrder("SKU1", 5));

        // Assert
        Assert.NotEqual(Guid.Empty, order.Id);
        repo.Verify(r => r.SaveAsync(order), Times.Once);
    }
}
```

22.3 Test Doubles

Double	Behaviour	Example
Dummy	Passed but never used	Fill parameter lists
Stub	Returns canned answers	Provide fixed data
Spy	Records calls for later verification	Verify email was sent
Mock	Stub + verification of expectations	Verify repo.Save called once
Fake	Working but simplified implementation	InMemoryDbContext

22.4 Mocking with Moq

```
var mock = new Mock();

// Setup return value
mock.Setup(r => r.GetAsync(It.IsAny()))
    .ReturnsAsync(new Order { Id = Guid.NewGuid(), Total = 100 });

// Verify a call was made with specific args
mock.Verify(r => r.SaveAsync(It.Is(o => o.Total == 100)), Times.Once);

// Verify no other calls
mock.VerifyNoOtherCalls();

// Throw on a call
mock.Setup(r => r.DeleteAsync(It.IsAny()))
    .ThrowsAsync(new NotFoundException("not found"));
```

22.5 What to Test (and What Not To)

- **Test behaviour, not implementation** — Tests that assert the order of private method calls break on refactor. Assert on inputs/outputs.
- **Do not test the framework** — Asserting that EF Core generates correct SQL is the framework team's job. Test that your LINQ expression returns expected results.
- **One assert per test (mostly)** — Each test should verify one behaviour. Multiple assertions on the same logical behaviour are OK.
- **Test names are documentation** — `MethodName_scenario_expected_result`. Avoid `Test1`, `Test2`.
- **Do not test getters/setters** — Trivial code does not need unit tests — they cost more to maintain than they save.

22.6 Integration Tests

```
// WebApplicationFactory boots the whole app in-process
public class OrdersApiTests : IClassFixture<> {
    private readonly HttpClient _client;
    public OrdersApiTests(WebApplicationFactory factory) {
        _client = factory.WithWebHostBuilder(b => {
            b.ConfigureServices(s => {
                s.RemoveAll<>();
                s.AddDbContext(o => o.UseSqlite("DataSource=:memory:"));
            });
        }).CreateClient();
    }

    [Fact]
    public async Task POST_creates_order_and_returns_201() {
        var resp = await _client.PostAsJsonAsync("/api/v1/orders",
            new { sku = "SKU1", qty = 5 });
        Assert.Equal(HttpStatusCode.Created, resp.StatusCode);
    }
}
```

23. Caching

Caching trades memory for speed. The right caching strategy can cut latency by 100× and database load by 90%. The wrong one causes stale data, consistency bugs, and out-of-memory crashes. .NET offers multiple layers, from in-process memory to distributed Redis.

23.1 Cache Layers

Layer	Pros	Cons / Notes
In-process (MemoryCache)	Fastest, no network	Lost on restart; not shared
Distributed (Redis, SQL)	Shared, persists restarts	Network latency (~1ms)
CDN (Cloudflare, Akamai)	Edge cache for static / cached content	Public only; cache-control headers
Browser	Per-user	Cache-Control, ETag, Last-Modified
Database (query cache, materialised views)	Same machine as data	Limited by DB engine

23.2 IMemoryCache

```
builder.Services.AddMemoryCache(o => o.SizeLimit = 1024 * 1024); // 1M entries

public class ProductService(IMemoryCache cache, IProductRepo repo) {
    public async Task GetAsync(Guid id) {
        return await cache.GetOrCreateAsync($"product:{id}", async entry => {
            entry.SetAbsoluteExpiration(TimeSpan.FromMinutes(10))
                .SetSlidingExpiration(TimeSpan.FromMinutes(2))
                .SetSize(1);
            return await repo.GetAsync(id);
        });
    }
}

public async Task InvalidateAsync(Guid id) {
    cache.Remove($"product:{id}");
}
```

23.3 IDistributedCache (Redis)

```
builder.Services.AddStackExchangeRedisCache(o => {
    o.Configuration = builder.Configuration.GetConnectionString("Redis");
    o.InstanceName = "myapp:";
});

public class ProductService(IDistributedCache cache) {
    public async Task GetAsync(Guid id) {
        var bytes = await cache.GetAsync($"product:{id}");
        if (bytes is not null)
            return JsonSerializer.Deserialize(bytes);

        var p = await repo.GetAsync(id);
        if (p is not null) {
```

```

        var json = JsonSerializer.SerializeToUtf8Bytes(p);
        await cache.SetAsync($"product:{id}", json,
            new DistributedCacheEntryOptions { AbsoluteExpirationRelativeToNow =
                TimeSpan.FromMinutes(10) });
    }
    return p;
}
}

```

23.4 Cache Patterns

Pattern	How	Trade-off
Cache-aside (lazy)	App checks cache; on miss, loads from DB and stores	Most common; explicit invalidation
Write-through	Write to cache + DB together	Strong consistency; slower writes
Write-behind	Write to cache; async to DB	Fast writes; risk of data loss
Refresh-ahead	Pre-populate before expiry	Smooths cold-cache hits

23.5 Expiration Policies

- **Absolute expiration** — Cache entry expires at a fixed time regardless of access. Use for data with a hard TTL (e.g., stock prices).
- **Sliding expiration** — Entry expires if not accessed within a window. Use for active sessions; bad for rarely-but-critically-needed data.
- **Both combined** — Set both: sliding window with a hard ceiling prevents long-lived sliding entries.
- **Token-based invalidation** — Tag cache entries with version tokens; bump the version on writes to invalidate all related entries at once.

23.6 Cache Stampede Protection

When a popular cache entry expires, hundreds of requests simultaneously miss and thundering-herd the database. Prevent with: (1) Mutex / SemaphoreSlim per key so only one request rebuilds while others wait, (2) stale-while-revalidate — return the stale value and refresh in the background, (3) probabilistic early expiration — add a small jitter so requests do not all expire at the same instant.

23.7 Output Caching & Response Caching

```

// Response caching (HTTP Cache-Control / ETag)
builder.Services.AddResponseCaching();
app.UseResponseCaching();

[ResponseCache(Duration = 60, Location = ResponseCacheLocation.Any, VaryByHeader =
    "Accept")]
public IActionResult Get() => Ok(...);

// Output caching (.NET 7+ - server-side, configurable)
builder.Services.AddOutputCache(o => {

```

```

        o.AddBasePolicy(b => b.Expire(TimeSpan.FromSeconds(30)));
        o.AddPolicy("Expire1m", b => b.Expire(TimeSpan.FromMinutes(1)).Tag("products"));
    });

    [HttpGet]
    [OutputCache(PolicyName = "Expire1m")]
    public IActionResult List() => Ok(...);

```

24. SignalR

SignalR is the ASP.NET Core library for real-time web communication: server pushing data to connected clients as it happens. It abstracts WebSockets (preferred) with Server-Sent Events and long-polling as fallbacks. Common use cases: chat, dashboards, live notifications, multiplayer games.

24.1 Hub Basics

```

// Server: define a Hub
public class ChatHub : Hub {
    public async Task SendMessage(string user, string message) {
        await Clients.All.SendAsync("ReceiveMessage", user, message);
    }
    public async Task SendToGroup(string group, string message) {
        await Clients.Group(group).SendAsync("ReceiveMessage",
Context.User!.Identity!.Name, message);
    }
    public override async Task OnConnectedAsync() {
        await Groups.AddToGroupAsync(Context.ConnectionId, "default");
        await base.OnConnectedAsync();
    }
}

// Program.cs
builder.Services.AddSignalR(o => o.EnableDetailedErrors =
builder.Environment.IsDevelopment());
app.MapHub("/chat");

```

24.2 Client (JavaScript)

```

// npm install @microsoft/signalr
const conn = new signalR.HubConnectionBuilder()
    .withUrl("/chat")
    .withAutomaticReconnect([0, 2000, 5000, 10000, 30000])
    .build();

conn.on("ReceiveMessage", (user, msg) => {
    appendMessage(user, msg);
});

await conn.start();
await conn.invoke("SendMessage", "alice", "Hello");

```

24.3 Targets for Sending

Target	Audience
Clients.All	All connected clients
Clients.Caller	Only the calling client
Clients.Others	All except the calling client
Clients.Group(g)	Members of group g
Clients.User(u)	All connections of a specific user (mapped via IAuthProvider)
Clients.Client(connId)	A specific connection by ID

24.4 Groups & Users

- **Groups** — Server-side tagging of connections. Add/Remove with `Groups.AddToGroupAsync(connId, name)`. Used for chat rooms, tenant-scoped updates.
- **Users** — Map connections to a user identifier (usually the auth claim sub). One user may have multiple connections (multiple tabs / devices). `Clients.User(userId)` reaches all of them.
- **OnConnectedAsync / OnDisconnectedAsync** — Override in the hub to manage group membership, log joins/leaves, or update presence.

24.5 Streaming

```
// Server: stream items to client
public async IAsyncEnumerable StreamStocks(string symbol,
    [FromQuery] CancellationToken ct) {
    while (!ct.IsCancellationRequested) {
        yield return _market.GetTick(symbol);
        await Task.Delay(1000, ct);
    }
}

// Client
const stream = conn.stream("StreamStocks", "AAPL");
stream.subscribe({
    next: tick => update(tick),
    complete: () => console.log("done"),
    error: err => console.error(err)
});
```

24.6 Scaling Out

In a single-instance deployment, SignalR holds connections in memory. To scale to multiple instances, you need a backplane so that a message sent from instance A reaches clients connected to instance B. Built-in backplanes: Redis, Azure SignalR Service. Azure SignalR Service is the recommended path in production — it offloads connection management entirely from the app.

24.7 Alternatives

- **WebSockets directly** — Lower-level, no abstraction. Use when you have a custom protocol or extreme performance needs.
- **gRPC streaming** — Bidirectional streaming for service-to-service real-time. Strongly typed; not browser-friendly without gRPC-Web.
- **Message queues** — For one-way server-to-client notifications at scale, a message broker (Kafka, RabbitMQ) plus a thin WebSocket layer is sometimes simpler.

25. Docker & Kubernetes Basics

Containers are the de-facto deployment unit for modern .NET services. Docker packages an app and its dependencies into a single image; Kubernetes orchestrates containers across a cluster — scheduling, scaling, healing, and rolling updates. Three years of .NET experience includes the ability to write a Dockerfile, build an image, push to a registry, and reason about a Deployment manifest.

25.1 Dockerfile for ASP.NET Core

```
# Multi-stage build: SDK to build, runtime to run
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["MyApp.csproj", "."]
RUN dotnet restore "MyApp.csproj" # restore cached by csproj change
COPY . .
RUN dotnet publish "MyApp.csproj" -c Release -o /app/publish /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080
USER app # non-root
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

Image-size tips: Use the `aspnet` runtime image (not `sdk`) for the final stage. For maximum size/startup wins, publish with `-p:PublishSingleFile=true -p:SelfContained=true` and use the `chiseled` base images (~100MB). For NativeAOT, target the `runtime-deps` image (~50MB total).

25.2 Essential Docker Commands

Command	Purpose
<code>docker build -t myapp:v1 .</code>	Build image from Dockerfile in current dir
<code>docker run -p 8080:8080 myapp:v1</code>	Run with port mapping
<code>docker run -e ConnectionStrings__Default=... myapp:v1</code>	Override config via env var
<code>docker ps / docker ps -a</code>	List running / all containers

Command	Purpose
<code>docker logs -f</code>	Follow container logs
<code>docker exec -it /bin/sh</code>	Shell into a running container
<code>docker images / docker rmi</code>	List / remove images
<code>docker system prune -a</code>	Remove unused images, containers, networks
<code>docker push / pull</code>	Push to / pull from a registry (Docker Hub, ACR, ECR)

25.3 Kubernetes Resources You Should Know

Resource	Purpose
Pod	Smallest unit; 1+ containers sharing network and volumes
Deployment	Manages replicas of a Pod; rolling updates, rollbacks
Service	Stable network name + load balancer in front of Pods
Ingress	HTTP/HTTPS routing from outside the cluster
ConfigMap / Secret	Configuration / sensitive data injected as env or files
StatefulSet	Like Deployment but with stable identity + storage (databases)
DaemonSet	One Pod per node (logging agent, node monitoring)
Job / CronJob	Run-to-completion / scheduled batch
HorizontalPodAutoscaler	Scale replicas based on CPU / custom metrics
Liveness / Readiness Probe	Health checks; liveness restarts, readiness removes from LB

25.4 Sample Deployment + Service

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels: { app: myapp }
  template:
    metadata:
      labels: { app: myapp }
    spec:
      containers:
        - name: app
          image: myregistry/myapp:v1
          ports: [{ containerPort: 8080 }]
          env:
            - name: ASPNETCORE_ENVIRONMENT

```

```

    value: Production
  resources:
    requests: { cpu: 100m, memory: 128Mi }
    limits: { cpu: 500m, memory: 512Mi }
  readinessProbe:
    httpGet: { path: /health/ready, port: 8080 }
    initialDelaySeconds: 5
    periodSeconds: 10
  livenessProbe:
    httpGet: { path: /health/live, port: 8080 }
    initialDelaySeconds: 15
    periodSeconds: 20
---
apiVersion: v1
kind: Service
metadata:
  name: myapp
spec:
  selector: { app: myapp }
  ports: [{ port: 80, targetPort: 8080 }]
  type: ClusterIP

```

26. Azure Basics

Microsoft Azure is the public cloud most .NET shops use. Three years of experience implies familiarity with App Service / Container Apps for hosting, Azure SQL Database or Cosmos DB for storage, Key Vault for secrets, Application Insights for observability, and Azure AD (Entra ID) for identity.

26.1 Compute Options for .NET

Service	What it is	Best For
App Service	PaaS web hosting — Windows/Linux	Web apps, APIs, simple ops
Container Apps	Server-sideless containers on KEDA	Microservices, autoscale
AKS	Managed Kubernetes	Complex microservices, full control
Functions	Serverless event-driven	Triggers (HTTP, queue, timer)
Container Instances	Single container, per-second billing	Batch jobs, sidecars
Service Fabric	Microservices runtime	Stateful services, legacy

26.2 Storage Options

Service	What it is	Best For
Azure SQL Database	Managed SQL Server	Relational, OLTP
Cosmos DB	Multi-model NoSQL (SQL, Mongo, Cassandra, Gremlin)	Globally distributed, low latency

Service	What it is	Best For
Blob Storage	Object storage (hot/cool/archive tiers)	Files, backups, media
Table Storage	Key-value NoSQL	Simple structured data at scale
Redis Cache	Managed Redis	Distributed cache, session store
Data Lake Storage	HDFS-compatible Blob	Big data analytics

26.3 Application Insights / OpenTelemetry

```
builder.Services.AddApplicationInsightsTelemetry(o => {
    o.ConnectionString = builder.Configuration["AppInsights:ConnectionString"];
    o.EnableAdaptiveSampling = true;
});

// Custom telemetry
public class OrderService(TelemetryClient tc) {
    public async Task CreateAsync(Order o) {
        var stopwatch = Stopwatch.StartNew();
        try {
            await repo.SaveAsync(o);
            tc.TrackEvent("OrderCreated", new Dictionary {
                ["OrderId"] = o.Id.ToString(), ["UserId"] = o.UserId
            });
            tc.TrackMetric("OrderProcessingTimeMs", stopwatch.ElapsedMilliseconds);
        }
        catch (Exception ex) {
            tc.TrackException(ex);
            throw;
        }
    }
}
```

26.4 Key Vault & Managed Identity

```
// Program.cs – use managed identity to read Key Vault (no secrets in code or config)
builder.Configuration.AddAzureKeyVault(
    new Uri($"https://{kvName}.vault.azure.net/"),
    new DefaultAzureCredential()); // works locally with az login, in cloud with MI

// Direct secret client
builder.Services.AddSingleton(sp =>
    new SecretClient(
        new Uri($"https://{kvName}.vault.azure.net/"),
        new DefaultAzureCredential()));
```

26.5 Common Patterns

- **Configuration** — App Service: app settings; Container Apps: env vars; Key Vault for secrets. Use Managed Identity so no connection strings in code.
- **Scaling** — App Service: scale up (tier) + out (instances). AKS: HPA. Container Apps: KEDA-based scaling on HTTP, CPU, queue length.
- **Deployment slots (App Service)** — Staging slot for warm-up + swap = zero-downtime deploy.

- **Traffic Manager / Front Door** — Global load balancer with routing rules (geo, weighted, performance).
- **Azure AD (Entra ID)** — Identity for both humans (login) and apps (service principals, managed identities). MSAL is the recommended auth library.

27. Multithreading

Multithreading is the explicit use of threads to parallelise work. In modern .NET, async/await covers IO-bound concurrency, while Task.Run, Parallel, and the TPL cover CPU-bound parallelism. Direct Thread usage are now rare except for very specific scenarios.

27.1 Thread vs Task vs ThreadPool

Concept	Backed By	Use For
Thread	OS thread, ~1MB stack	Long-running, dedicated
ThreadPool thread	Reusable worker from pool	Short tasks, async continuations
Task	Abstraction over async work	Composition (await, WhenAll)
ValueTask	Lightweight Task (struct)	Hot path, often synchronous

27.2 Thread Safety Primitives

Primitive	Mechanism	Use Case
lock / Monitor	Mutual exclusion	Critical sections
Monitor.TryEnter	lock with timeout	Avoid deadlocks
Mutex	Cross-process lock	Single instance across processes
SemaphoreSlim	Counting, async-friendly	Limit concurrency (e.g., 5 parallel HTTP)
ReaderWriterLockSlim	Multiple readers OR one writer	Read-heavy caches
Interlocked	Atomic int ops (incr, exchange, compareExchange)	Counters, flags
volatile	Prevents reordering	Flags, not full safety
ConcurrentDictionary	Lock-free / striped	Shared dictionary
Channel	Producer/consumer queue	Pipelines

27.3 lock Statement

```
public class Counter {
    private readonly object _gate = new();
    private int _value;

    public void Increment() {
        lock (_gate) { // only one thread at a time
            _value++;
        }
    }
    public int Read() {
        lock (_gate) { return _value; }
    }
}

// For simple counters, prefer Interlocked:
private int _value;
public void Increment() => Interlocked.Increment(ref _value);
public int Read() => Interlocked.CompareExchange(ref _value, 0, 0);
```

Lock hygiene: Never lock on `this`, a Type object, or a string literal — these are reachable from outside your class and can deadlock. Always use a private readonly object as the gate. Never await inside a lock — use `SemaphoreSlim` instead.

27.4 SemaphoreSlim (async-friendly)

```
public class ThrottledHttpClient {
    private readonly SemaphoreSlim _gate = new(5); // max 5 concurrent
    private readonly HttpClient _http;

    public async Task GetAsync(string url) {
        await _gate.WaitAsync();
        try {
            return await _http.GetStringAsync(url);
        }
        finally {
            _gate.Release();
        }
    }
}
```

27.5 Parallel / PLINQ

```
// Parallel.ForEach: blocking, CPU-bound parallelism
var options = new ParallelOptions { MaxDegreeOfParallelism =
    Environment.ProcessorCount };
Parallel.ForEach(items, options, item => {
    Process(item);
});

// PLINQ: parallel LINQ
var results = items
    .AsParallel()
    .AsOrdered()
    .WithDegreeOfParallelism(8)
    .Select(ExpensiveComputation)
    .ToList();
```

```
// For IO-bound work, prefer async + WhenAll (NOT Parallel.ForEach)
await Task.WhenAll(urls.Select(async url => await DownloadAsync(url)));
```

27.6 Channels

```
var channel = Channel.CreateBounded(100);

// Producer
_ = Task.Run(async () => {
    foreach (var url in urls) {
        await channel.Writer.WriteAsync(url);
    }
    channel.Writer.Complete();
});

// Consumers (3 of them)
var consumers = Enumerable.Range(0, 3).Select(async _ => {
    await foreach (var url in channel.Reader.ReadAllAsync()) {
        await DownloadAsync(url);
    }
});
await Task.WhenAll(consumers);
```

27.7 Deadlock Causes & Prevention

- **Lock ordering** — Two threads each acquire locks in opposite order → circular wait. Fix: always acquire locks in the same global order.
- **Awaiting inside a lock** — The continuation may run on a different thread that cannot re-acquire the lock → deadlock. Fix: use `SemaphoreSlim.WaitAsync`.
- **Blocking on async code** — `.Result` or `.Wait()` in UI/legacy ASP.NET deadlocks. Fix: await all the way up; or `.ConfigureAwait(false)` in library code.
- **Single-threaded SynchronizationContext** — UI thread blocked waiting for an async operation whose continuation needs the UI thread. Fix: `ConfigureAwait(false)` in libraries, or move blocking work to `Task.Run`.

28. Performance Optimization

Performance work is iterative: measure → diagnose → optimise → verify. Without measurement, optimisations are guesses. .NET ships with rich tooling (`dotnet-counters`, `dotnet-trace`, `BenchmarkDotNet`, `PerfView`) and many framework features that improve performance with no code change.

28.1 Measure First

Tool	Use For
<code>dotnet-counters</code>	Real-time CPU, GC, ThreadPool, HTTP counters
<code>dotnet-trace</code>	ETW events → speedscope / Chromium trace
<code>dotnet-dump</code>	Heap dump for memory analysis

Tool	Use For
PerfView	CPU sampling, GC details, allocation tracking
BenchmarkDotNet	Micro-benchmark framework with statistical rigour
Visual Studio Profiler	CPU usage, memory usage, async profiling
Application Insights	Production telemetry, APM
dotMemory / dotTrace (JetBrains)	Memory + CPU profilers

28.2 BenchmarkDotNet Example

```
[MemoryDiagnoser] // capture allocations + GC counts
public class StringJoinBench {
    private readonly string[] _parts = Enumerable.Range(0, 100).Select(i =>
        i.ToString()).ToArray();

    [Benchmark(Baseline = true)]
    public string Concat() => string.Concat(_parts);

    [Benchmark]
    public string Join() => string.Join(",", _parts);

    [Benchmark]
    public string StringBuilder() {
        var sb = new StringBuilder();
        foreach (var p in _parts) sb.Append(p).Append(',');
        return sb.ToString();
    }
}
// dotnet run -c Release → Statistical results in a table
```

28.3 Common Optimisations

- **AsNoTracking on read-only EF queries** — Halves allocations and time.
- **Span / stackalloc** — Avoid heap allocation for temporary buffers in hot paths.
- **ArrayPool** — Rent arrays instead of allocating; reuse across calls.
- **String pool / interning** — For repeated identical strings, use string.Intern or a shared dictionary.
- **Object pool** — Pool expensive-to-create objects (DB connections, large buffers) via ObjectPool.
- **Escape analysis** — Allocate small short-lived objects freely — Gen 0 GC is cheap. Do NOT pre-optimize with structs unless measurements demand it.
- **Lazy** — Defer construction of rarely-used heavy members.
- **Compiled queries (EF Core)** — Cache the query plan for hot repeated queries.
- **Output caching** — Cache full HTTP responses at the server or CDN.

28.4 Allocation-Aware Code

```
public readonly struct Point { public int X, Y; } // struct, no allocation

// Bad: boxing
object o = 42; // allocates box
```

```
// Good: generic, no boxing
T Identity(T t) where T : struct => t;

// Bad: LINQ on hot path
for (int i = 0; i < arr.Length; i++) {
    if (arr.Where(x => x > 0).Count() > 5) { /* ... */ } // allocates iterator
}

// Good: manual loop
for (int i = 0; i < arr.Length; i++) {
    int count = 0;
    for (int j = 0; j < arr.Length; j++) if (arr[j] > 0) count++;
    if (count > 5) { /* ... */ }
}
```

28.5 Async Performance

- **ConfigureAwait(false)** — In library code, avoids capturing SynchronizationContext and saves ~50ns per await.
- **ValueTask** — Reduces allocation when the result is often available synchronously.
- **Avoid async/await in tight loops** — Use Task.WhenAll over a list instead of awaiting in a loop.
- **SetMinThreads** — Avoids the 1-thread-per-second ramp-up under burst load.
- **Channel** — For producer-consumer pipelines; faster than BlockingCollection.
- **HttpClient factory** — Reuses SocketsHttpHandler connection pool; avoids socket exhaustion from `new HttpClient()` per request.

29. Security Best Practices

Security is everyone's job. A three-year developer should be able to identify the OWASP Top 10 in their own code, know the basics of authentication and authorisation, handle secrets correctly, and write code that fails safe under unexpected input.

29.1 OWASP Top 10 (2021)

Risk	Example	Mitigation
A01 Broken Access Control	Missing authorisation checks; IDOR	[Authorize] + resource ownership
A02 Cryptographic Failures	Weak crypto, hardcoded keys, no TLS	AES-GCM, BCrypt/Argon2, TLS 1.2+
A03 Injection	SQL/NoSQL/LDAP injection	Parameterised queries, EF Core LINQ
A04 Insecure Design	No threat modelling, missing rate limits	Threat model, defence in depth
A05 Security Misconfiguration	Default creds, verbose errors, CORS *	Lock down, hide stack traces
A06 Vulnerable Components	Outdated NuGet packages with CVEs	dotnet list package --vulnerable
A07 Auth Failures	Brute force, weak session IDs	Rate limit, MFA, secure cookies

Risk	Example	Mitigation
A08 Data Integrity Failures	Unsigned JWTs, untrusted deserialisation	Sign tokens, validate inputs
A09 Logging Failures	No audit logs for security events	Log logins, failures, admin ops
A10 SSRF	Server fetches user-supplied URLs	Whitelist outbound domains

29.2 SQL Injection — Always Parameterise

```
// x VULNERABLE – string concatenation
var sql = $"SELECT * FROM Users WHERE Email = '{email}'";
await db.Database.ExecuteSqlRawAsync(sql);

// ✓ Safe – parameterised
await db.Users
    .FromSqlInterpolated($"SELECT * FROM Users WHERE Email = {email}")
    .ToListAsync();

// ✓ Safe – Dapper with parameters
await conn.QueryAsync(
    "SELECT * FROM Users WHERE Email = @email",
    new { email });
```

29.3 XSS — Encode Output

- **Razor @ expression** — HTML-encodes by default. Safe for inline content.
- **@Html.Raw** — Bypasses encoding — only on trusted content (or sanitised via a library like Ganss.Html).
- **Content Security Policy** — Response header that whitelists script sources. Strong defence even if XSS slips in.
- **React / Angular** — Frameworks auto-escape; avoid dangerouslySetInnerHTML / [innerHTML] on user content.

29.4 CSRF

- **Anti-forgery tokens** — Server embeds a token in the form; validates on POST. ASP.NET Core: [ValidateAntiForgeryToken] or auto-antiforgery on forms.
- **SameSite cookies** — Set SameSite=Lax or Strict on auth cookies — browser refuses to send them on cross-site POST.
- **Bearer tokens in JS apps** — CSRF is not an issue if auth is via Authorization header (not cookies) — the browser does not auto-attach it.

29.5 Password Storage

```
using BCrypt.Net;

// Hash with work factor 12 (default) – never MD5/SHA alone
string hash = BCrypt.HashPassword(password, workFactor: 12);

// Verify
bool valid = BCrypt.Verify(password, hash);
```

```
// Modern alternative: Argon2id via Konscious.Security.Cryptography.Argon2
// or ASP.NET Core Identity v3+ which uses PBKDF2 with HMAC-SHA256 + 600k iterations
```

29.6 Secrets Management

Environment	Approach
Local dev	appsettings.Development.json (excluded from source control) OR dotnet user-secrets
Production (Azure)	Key Vault + Managed Identity (no secrets in code/config)
Production (AWS)	Secrets Manager + IAM role
Production (K8s)	Secrets (base64) — or external Sealed Secrets / Vault
NEVER	Plain text in appsettings.json committed to git

29.7 HTTPS & TLS

- **Use HSTS** — `app.UseHsts()` in production — forces HTTPS for subdomain visitors.
- **Redirect HTTP** — `app.UseHttpsRedirection()` — sends 307 to HTTPS.
- **TLS 1.2 minimum** — TLS 1.0 / 1.1 are deprecated. .NET 8 defaults to TLS 1.2/1.3.
- **Certificate management** — Let's Encrypt for self-host; Azure App Service Managed Certificates for PaaS; cert-manager for K8s.

30. REST & HTTP Concepts

A solid grasp of HTTP and REST is mandatory for any web developer. Beyond verbs and status codes, you should understand headers, content negotiation, caching, idempotency, and the trade-offs of REST vs alternatives like gRPC and GraphQL.

30.1 HTTP Request/Response Anatomy

```
POST /api/orders HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGc...
Content-Type: application/json
Accept: application/json
Idempotency-Key: 9f3a-...
Content-Length: 64

{"sku": "ABC", "qty": 5}

---

HTTP/1.1 201 Created
Location: /api/orders/9f3a
Content-Type: application/json
ETag: "v1"
```

```
Cache-Control: private, max-age=60
{"id": "9f3a", "sku": "ABC", "qty": 5}
```

30.2 Important Headers

Header	Purpose	Use
Authorization	Bearer token / Basic auth	Auth
Accept	Acceptable response media types	Content negotiation
Content-Type	Body media type	Request format
Cache-Control	Caching directives (max-age, no-cache, public)	Caching
ETag	Version identifier	Conditional requests
If-None-Match	Send ETag; server returns 304 if unchanged	Conditional GET
Location	URI of newly created resource	201 response
Idempotency-Key	Unique ID for safe retry	POST idempotency
X-Forwarded-For	Original client IP (behind proxy)	Networking
X-Correlation-Id	Distributed tracing ID	Observability

30.3 Status Code Categories

Category	Examples
1xx Informational	Rarely used (HTTP/2 uses for protocol upgrade)
2xx Success	200 OK, 201 Created, 204 No Content, 206 Partial
3xx Redirection	301 Moved, 304 Not Modified
4xx Client Error	400 Bad Request, 401/403 Auth, 404/409, 422, 429 Too Many
5xx Server Error	500 Internal, 502 Bad Gateway, 503 Unavailable, 504 Timeout

30.4 Idempotency

- **Idempotent methods** — GET, PUT, DELETE, HEAD — calling them multiple times has the same effect as calling once.
- **Non-idempotent** — POST and PATCH by default. Use Idempotency-Key header so the server can deduplicate retries of the same logical request.
- **Implementation** — Server stores Idempotency-Key → response for 24h. On retry, return the cached response instead of re-executing.

30.5 REST Maturity (Richardson Model)

Level	Characteristics	Notes
Level 0	One endpoint, all actions via POST	POX / RPC over HTTP
Level 1	Resource URIs /orders/42	Nouns, not verbs
Level 2	HTTP verbs + status codes	Most "REST" APIs live here
Level 3	HATEOAS — responses include links	Hypermedia-driven; rare in practice

30.6 REST vs gRPC vs GraphQL

Style	Characteristics	Best For
REST	HTTP/JSON, ubiquitous, human-readable	Public APIs, simple resources
gRPC	HTTP/2 + Protobuf, strongly typed, bidirectional streaming	Internal service-to-service, high throughput
GraphQL	Single endpoint, client queries shape	Mobile apps, many views, aggregation
WebSockets	Persistent connection, full-duplex	Real-time, chat, push

30.7 Pagination Patterns

- **Offset/Limit** — ?page=2&size=20 — simple but skips items inserted between requests. Bad for high-churn data.
- **Cursor** — ?after=abc123&limit=20 — stable, indexed; returns a cursor for the next page.
- **Keyset** — ?since=2026-01-01 — use a sortable column as the cursor.
- **Headers vs body** — Return total count in a header (X-Total-Count) to avoid polluting the body.

31. System Design Basics

System design interviews test whether you can reason about scaling, trade-offs, and architecture. Three years of experience expects you to handle mid-level design questions: design a URL shortener, a rate limiter, a notification service, or an image upload pipeline. The goal is not a perfect design but a clear thought process.

31.1 Six-Step Framework

Step	What to Do
1. Clarify requirements	Functional + non-functional; scale, latency, availability

Step	What to Do
2. Estimate capacity	QPS, storage, bandwidth, connections
3. High-level design	Box diagram: clients, services, data stores, queues
4. Deep dive	Pick 1-2 components and detail them
5. Scale it	Caching, sharding, replicas, async, CDN
6. Trade-offs	CAP, consistency, complexity, cost

31.2 Key Concepts

- **Vertical vs horizontal scaling** — Scale up = bigger machine; scale out = more machines. Horizontal removes the single-machine ceiling.
- **Load balancer** — Distributes traffic across instances. L4 (TCP) vs L7 (HTTP).
- **Caching layers** — Browser, CDN, reverse proxy (Nginx), app (MemoryCache), distributed (Redis), DB (buffer pool).
- **Database replication** — Primary for writes, replicas for reads. Async replication has lag; sync has latency cost.
- **Sharding** — Horizontal partition by key (user id, geography). Adds complexity: cross-shard queries, rebalancing.
- **CAP theorem** — In a network partition, choose consistency OR availability. CP systems (e.g., HBase) vs AP systems (e.g., Cassandra).
- **Eventual consistency** — Acceptable for many use cases (likes, feed); unacceptable for billing.
- **Backpressure** — When a downstream is slow, signal upstream to slow down. Use bounded channels, queues with limits.

31.3 Worked Example: Design a URL Shortener

Walk through the six steps for "design a URL shortener like bit.ly":

- **Step 1 — Requirements** — Functional: shorten a URL; redirect short URL to original. Non-functional: 100M shortenings/day, 10B redirects/day, 99.9% availability, <100ms redirect latency.
- **Step 2 — Capacity** — 100M new URLs/day × 5 years = ~180B rows. Each row ~500 bytes → ~90TB storage. Reads 100× writes → 10B redirects/day × ~50 bytes response = 500GB/day egress.
- **Step 3 — High-level** — Client → API (shorten) → DB. Client → Redirect service → Cache (Redis) → DB → 301 to original.
- **Step 4 — Deep dive (ID generation)** — Option A: auto-increment in DB (bottleneck). Option B: UUID (long). Option C: base62 encoding of a counter from a Snowflake-like ID service (timestamp + machine + sequence) — short, sortable, distributed.
- **Step 5 — Scale** — Read-heavy: 99% cache hit on Redis. CDN for popular short URLs. DB: 1 primary + N read replicas. Sharding by short code prefix if needed.
- **Step 6 — Trade-offs** — 301 (cacheable, browser caches) vs 302 (always hits service for analytics). Custom aliases vs random (collision risk).

33. Coding Problems

Coding rounds typically ask for two patterns: data structures (reverse a linked list, valid parentheses, two-sum) and small domain implementations (a rate limiter, a thread-safe cache, an async producer-consumer). Below are sample problems with model answers in idiomatic C#.

33.1 Two Sum

```
// Given an array and a target, return indices of two numbers that add up to target.
public int[] TwoSum(int[] nums, int target) {
    var seen = new Dictionary(); // value → index
    for (int i = 0; i < nums.Length; i++) {
        int need = target - nums[i];
        if (seen.TryGetValue(need, out int j))
            return new[] { j, i };
        seen[nums[i]] = i;
    }
    throw new ArgumentException("No solution");
}
// O(n) time, O(n) space
```

33.2 Reverse a Linked List

```
public class ListNode { public int Val; public ListNode? Next; }

public ListNode? Reverse(ListNode? head) {
    ListNode? prev = null, curr = head;
    while (curr is not null) {
        var next = curr.Next;
        curr.Next = prev;
        prev = curr;
        curr = next;
    }
    return prev; // new head
}
// O(n) time, O(1) space
```

33.3 Valid Parentheses

```
public bool IsValid(string s) {
    var stack = new Stack();
    var pairs = new Dictionary {
        [')'] = '(', [']'] = '[', ['}'] = '{'
    };
    foreach (var c in s) {
        if (pairs.ContainsValue(c)) stack.Push(c); // opener
        else if (pairs.TryGetValue(c, out var opener)) { // closer
            if (stack.Count == 0 || stack.Pop() != opener)
                return false;
        }
    }
    return stack.Count == 0;
}
```

33.4 Async Producer-Consumer with Channels

```
public class JobProcessor {
    private readonly Channel _channel =
        Channel.CreateBounded(100);

    public async Task EnqueueAsync(Job job, CancellationToken ct = default)
        => await _channel.Writer.WriteAsync(job, ct);

    public async Task RunAsync(int workers, CancellationToken ct) {
        var tasks = Enumerable.Range(0, workers).Select(_ => WorkerAsync(ct));
        await Task.WhenAll(tasks);
    }

    private async Task WorkerAsync(CancellationToken ct) {
        await foreach (var job in _channel.Reader.ReadAllAsync(ct)) {
            try { await ProcessAsync(job, ct); }
            catch (Exception ex) { _logger.LogError(ex, "Job failed"); }
        }
    }
}
```

33.5 Thread-Safe LRU Cache

```
public class LruCache where K : notnull {
    private readonly int _capacity;
    private readonly LinkedList<(K key, V value)> _list = new();
    private readonly Dictionary<> _map = new();
    private readonly object _gate = new();

    public LruCache(int capacity) => _capacity = capacity;

    public bool TryGet(K key, out V? value) {
        lock (_gate) {
            if (!_map.TryGetValue(key, out var node)) { value = default; return false; }

            _list.Remove(node); _list.AddFirst(node); // mark as recently used
            value = node.Value.value; return true;
        }
    }

    public void Set(K key, V value) {
        lock (_gate) {
            if (_map.TryGetValue(key, out var node)) {
                _list.Remove(node);
            }
            else if (_map.Count >= _capacity) {
                var last = _list.Last!; _list.RemoveLast();
                _map.Remove(last.Value.key);
            }
            _list.AddFirst((key, value));
            _map[key] = _list.First!;
        }
    }
}
```

34. HR & Scenario-Based Questions

HR and behavioural rounds test communication, conflict resolution, ownership, and cultural fit. Use the STAR framework (Situation, Task, Action, Result) for every answer. Prepare three concrete stories from your experience: a hard bug, a conflict with a teammate, and a project you led.

34.1 STAR Framework

Letter	What to Cover	Example
Situation	Set the context in 1-2 sentences	At Acme, our checkout API was timing out during sales
Task	What was your responsibility?	I was the lead backend dev on the checkout team
Action	What did YOU do? (not "we")	I profiled the hot path, found N+1 EF queries, replaced with a single JOIN, added Redis caching
Result	Quantified outcome	P99 latency dropped from 4.2s to 220ms; checkout conversion +8%

34.4 Questions to Ask the Interviewer

- What is the most recent technical decision the team reversed? Why?
- How are code reviews handled? What is the typical turnaround?
- What does on-call look like — frequency, severity of pages, fallback support?
- What is the deployment pipeline — frequency, rollback procedure, automated tests?
- What is the team's biggest technical debt? What is the plan to address it?
- How does the team balance feature work with technical investment?
- What does success look like for this role in the first 6 months?

35. Quick Revision Notes

A condensed cheat-sheet for the last 30 minutes before the interview. Each line is a one-liner you should be able to expand into a 60-second answer.

35.1 C# & Runtime

- string is immutable; use StringBuilder for loops; string.Create for high-perf ASCII.
- var is compile-time inference; dynamic is runtime (avoid except interop).
- async/await is for IO; Task.Run is for CPU. Never wrap IO in Task.Run.
- Never call .Result or .Wait() on async — deadlock or starvation.
- Always pass CancellationToken through async APIs.
- Dispose deterministic; finalise non-deterministic. Prefer SafeHandle to finalisers.
- Span = stack-only slice of memory; no allocation; never store in fields.

- record = reference type with value equality + with expressions; DTOs made simple.
- delegate = type-safe function ptr; event = encapsulated delegate (only owner fires).
- LINQ deferred until enumerated; ToList/Count/First trigger it; IQueryable composes SQL.
- GC: Gen 0 short-lived, Gen 2 long-lived, LOH $\geq$ 85KB; Server GC for ASP.NET Core.
- Tiered compilation: JIT twice (fast then optimised); dynamic PGO in .NET 6+.

35.2 ASP.NET Core

- Middleware order: ExceptionHandler → HSTS → HTTPS → Static → Routing → CORS → Auth → Authz → Endpoints.
- DI lifetimes: Singleton (app), Scoped (request), Transient (always new).
- Never inject Scoped into Singleton — captive dependency.
- DbContext is scoped; inject IDbContextFactory for background services.
- Use [ApiController] for auto-400, model binding inference, ProblemDetails.
- AsNoTracking() for read-only queries — ~2× faster, half the memory.
- Polly for retries / circuit breaker / timeout; HttpClientFactory for connection pool.
- Use health checks (/health/live vs /health/ready) for K8s probes.
- Rate-limiter middleware in .NET 7+ (token bucket, fixed/sliding window).
- Output caching in .NET 7+ (server-side); Response caching (HTTP Cache-Control).

35.3 EF Core

- Code-first with migrations: dotnet ef migrations add Name; dotnet ef database update.
- Include() for eager load; AsSplitQuery() for multiple Includes on collections.
- Avoid lazy loading → N+1. Use Select() to project to DTOs.
- [Timestamp] / RowVersion for optimistic concurrency; catch DbUpdateConcurrencyException.
- ExecuteUpdate / ExecuteDelete (.NET 7+) for bulk ops without loading entities.
- Compiled queries for hot repeated paths; AsNoTracking for reads.
- Transaction: BeginTransactionAsync → SaveChanges → CommitAsync.

35.4 Concurrency & Threading

- lock (gate) {} — never on this/Type/string; use private readonly object.
- SemaphoreSlim.WaitAsync — async-friendly lock; never await inside lock.
- Interlocked.Increment for atomic counters; volatile for flags (not full safety).
- ConcurrentDictionary for shared map; Channel for producer-consumer.
- Task.WhenAll (parallel), Task.WhenAny (race); Task.WhenEach (.NET 9) streams.
- Parallel.ForEach for CPU-bound; Task.WhenAll + Select for IO-bound.
- ConfigureAwait(false) in library code; ignore in ASP.NET Core (no-op).
- Deadlock causes: lock ordering, await in lock, .Result in UI/legacy ASP.NET.

35.5 Web & API

- REST verbs: GET (safe, idempotent), PUT (idempotent), POST (neither), DELETE (idempotent).
- Status: 200 OK, 201 Created (+ Location), 204 No Content, 400 Bad Request, 401 vs 403, 404, 409, 422, 429, 500, 503.
- 401 = who are you? 403 = I know you, but you can't.
- Idempotency-Key header for safe POST retries.
- Pagination: cursor-based > offset/limit for high-churn data.
- ETag + If-None-Match → 304 Not Modified; Cache-Control for client/proxy caching.
- JWT: header.payload.signature; verify signature + exp; short-lived; refresh tokens.
- CORS preflight (OPTIONS) for non-simple requests; allow credentials cautiously.

35.6 Patterns & SOLID

- Singleton: Lazy or DI AddSingleton; thread-safe by default.
- Strategy: inject algorithm behind interface; no if/else switch on type.
- Repository + Unit of Work: DbContext already is one — wrap only when needed.
- CQRS: split reads (queries) from writes (commands); MediatR is the popular lib.
- Circuit Breaker (Polly): trip on N failures, half-open after cooldown, reset on success.
- Outbox pattern: DB write + event publish atomically; background worker publishes.
- Saga: distributed tx as sequence of local txns + compensations on failure.
- SRP: one reason to change. OCP: extend, don't modify. LSP: subtypes substitute. ISP: small interfaces. DIP: depend on abstractions.

35.7 DevOps & Cloud

- Multi-stage Dockerfile: SDK to build, aspnet:8.0 to run; chiseled for size.
- K8s: Deployment (replicas) + Service (network) + Ingress (HTTP). Probes: live (restart) vs ready (LB).
- HPA scales on CPU / custom metrics; VPA for sizing.
- Azure: App Service (PaaS), AKS (K8s), Container Apps (serverless containers), Functions (events).
- Managed Identity + Key Vault — no secrets in code/config.
- Application Insights / OpenTelemetry for traces, metrics, logs.
- CI/CD: GitHub Actions / Azure DevOps; build → test → publish → deploy; slots for zero-downtime.

35.8 Final Tips

- Think aloud during coding rounds; silent candidates look stuck.
- Ask clarifying questions before writing code; constraints matter.
- Start with brute force, then optimise; do not jump to the optimal solution first.
- Test your code with edge cases before declaring "done".
- For system design, draw the box diagram FIRST, then deep-dive one component.
- Quantify your impact in behavioural answers (X%, Y ms, Z users).

- Have 2-3 STAR stories ready: a hard bug, a conflict, a leadership moment.
- Prepare 3-5 thoughtful questions to ask the interviewer.
- Sleep well the night before. A rested brain > last-minute cramming.

Final reminder: Three years of experience means you should be able to talk about trade-offs, not just definitions. Whenever an interviewer asks "what is X?", answer with what it is, when to use it, and one limitation. The third sentence is what separates mid-level from junior.